

算法板子合并版

算法板子合并版

一、动态规划

1. 数位 DP
 - 1.1 数字计数
2. 线性 DP
 - 2.1 最长上升子序列 I
 - 2.2 最长上升子序列 II

二、数据结构

1. 前缀与差分类
 - 1.1 前缀置换
 - 1.2 前缀矩阵积
 - 1.3 前缀和 + 等差数列
 - 1.4 前缀和 + 平方数列
 - 1.5 前缀和 + 多项式
 - 1.6 高阶前缀和 / 差分 (NTT)
 - 1.7 SOS DP (子集和变换)
2. 区间数据结构
 - 2.1 线段树 (区间乘、区间加、区间和、平方和)
 - 2.2 树状数组 1: 单点加 / 区间和
 - 2.3 树状数组 2: 区间加 / 单点查
 - 2.4 树状数组处理二维偏序
3. 倍增与 RMQ
 - 3.1 ST 表
 - 3.2 倍增表
4. 其他常用结构
 - 4.1 三维前缀和
 - 4.2 字符串哈希 (双模)
 - 4.3 最小表示法
 - 4.4 rope
 - 4.5 ordered_set
 - 4.6 普通可换根 Trie (统计前缀数量版)

三、图论

1. 树的重心
2. Bellman-Ford
3. SPFA
 - 3.1 SPFA 求最短路
 - 3.2 SPFA 判断负环
4. Floyd
 - 4.1 Floyd 求最短路
5. 二分图
 - 5.1 染色法判断二分图
 - 5.2 二分图的最大匹配
6. 传递闭包
7. 树的直径
 - 7.1 核心结论
 - 7.2 两次 DFS / BFS
 - 7.3 树形 DP 求直径
 - 7.4 树的中心
 - 7.5 两棵树合并后的最小直径

- 8. 最近公共祖先 LCA
 - 8.1 倍增
 - 8.2 Tarjan 离线 LCA
- 9. 最小生成树
 - 9.1 Kruskal
 - 9.2 Prim
 - 9.3 Boruvka
- 10. 树形换根 DP
- 11. 缩点

四、数学知识

- 1. 数论
 - 1.1 筛质数
 - 埃氏筛
 - 线性筛
 - 1.2 约数个数
 - 1.3 约数之和
 - 1.4 阶乘分解
 - 1.5 gcd / exgcd
 - 最大公约数
 - 扩展欧几里得
 - 扩展欧拉定理
 - 1.6 数论分块
 - 向下取整分块
 - 向上取整分块
 - 1.7 Legendre 定理
 - 1.8 威尔逊定理
 - 1.9 筛质数 + 求 μ
 - 1.10 矩阵快速幂
 - 普通矩阵幂
 - min-plus 矩阵快速幂
 - 1.11 支持除 0 的模数
- 2. 线性代数
 - 2.1 高斯消元 (实数域)
 - 2.2 高斯消元 (异或方程组)
 - 2.3 线性基
 - 标准模板
 - 带路径还原
 - 线性基合并
- 3. 博弈论
 - 3.1 SG 函数
 - 3.2 集合 Nim
 - 3.3 拆分 Nim
- 4. 组合数学
 - 4.1 组合数
 - 杨辉三角
 - $O(n^2)$ 求组合数
 - 阶乘逆元预处理
 - 4.2 Lucas 定理
 - 4.3 插板法
 - 4.4 多重集合排列
 - 4.5 第二类斯特林数
 - 4.6 分拆数
 - 4.7 范德蒙德卷积

- 4.8 错位排列
- 4.9 卡特兰数
- 括号序列计数常用写法

5. 其他常用内容

- 5.1 分数结构体
- 5.2 C++ 常用数学函数
- 5.3 生成函数

五、计算几何

1. 基础模板

- 1.1 浮点数安全比较
- 1.2 点、向量、叉积、点积
- 1.3 三点叉积与方向判断
- 1.4 求两直线交点

2. 基础判断

- 2.1 点在线段上
- 2.2 点在多边形内

3. 面积公式

- 3.1 三角形面积
- 3.2 多边形面积

4. 凸包

- 4.1 Andrew 凸包

5. 旋转卡壳

- 5.1 求凸包直径

6. 极角排序

- 6.1 适用场景
- 6.2 常用写法

7. 题目型模板

- 7.1 CF993A Two Squares

8. 圆的基础内容

- 8.1 圆的基本定义
- 8.2 点和圆的位置关系
- 8.3 点在直线上的投影
- 8.4 点到直线距离
- 8.5 直线和圆的位置关系
- 8.6 圆和圆的位置关系
- 8.7 直线与圆的交点
- 8.8 圆与圆的交点
- 8.9 圆面积与周长
- 8.10 扇形面积

六、杂板子整理

- 1. 三分
- 2. 对拍随机数
- 3. 对拍器

七、贪心

1. 区间问题

- 1.1 区间合并
- 1.2 区间选点
- 1.3 最大不相交区间数量
- 1.4 区间分组
- 1.5 区间覆盖

2. 反悔贪心

八、数值处理

- 1. `__int128`

- 2. long double
- 3. 高精度
- 4. 文件读入

一、动态规划

1. 数位 DP

1.1 数字计数

适用场景

求区间 $[a, b]$ 中每个数字 $0 \sim 9$ 出现的总次数。

常见转化：

- 先求 $[1, x]$ 中每个数码出现次数；
- 再用 $\text{calc}(b) - \text{calc}(a - 1)$ 得到答案。

核心思路

设：

- $\text{ten}[i] = 10^i$
- $\text{dp}[i]$ 表示在所有 i 位数的低 i 位贡献中，某个数字在每一位出现的总次数贡献

预处理：

$$\text{dp}[i] = i \cdot 10^{i-1}$$

从高位到低位枚举当前位，统计：

- 当前位左边固定时，右边自由变化带来的整段贡献；
- 当前位取比原数当前位更小的值时带来的贡献；
- 当前位恰好等于原数当前位时，右侧剩余部分形成的贡献；
- 额外减去前导零贡献。

复杂度

- 时间复杂度： $O(\log_{10} n)$
- 空间复杂度： $O(\log_{10} n)$

注意点

- 区间统计统一写成：`calc(b) - calc(a - 1)`。
- 统计 `0` 时一定要处理前导零，模板里通过 `cnt[0] -= ten[i - 1]` 消掉非法贡献。
- 当 `a = 1` 时，`a - 1 = 0`，要保证分解数字时不会出错。
- 数据范围到 10^{12} 时，使用 `long long`。

模板

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;

const int N = 20;

ll ten[N], dp[N];
int num[N];

array<ll, 10> calc(ll x) {
    array<ll, 10> cnt{};
    if (x <= 0) return cnt;

    int len = 0;
    while (x) {
        num[++len] = x % 10;
        x /= 10;
    }

    for (int i = 1; i <= 15; i++) {
        ten[i] = ten[i - 1] * 10;
        dp[i] = 1LL * i * ten[i - 1];
    }

    for (int i = len; i >= 1; i--) {
        for (int d = 0; d <= 9; d++) {
            cnt[d] += dp[i - 1] * num[i];
        }
        for (int d = 0; d < num[i]; d++) {
            cnt[d] += ten[i - 1];
        }
        ll suf = 0;
        for (int j = i - 1; j >= 1; j--) {
            suf = suf * 10 + num[j];
        }
        cnt[num[i]] += suf + 1;
        cnt[0] -= ten[i - 1];
    }

    return cnt;
}

void solve() {
    ten[0] = 1;
    ll a, b;
    cin >> a >> b;

    auto A = calc(a - 1);
    auto B = calc(b);

    for (int i = 0; i <= 9; i++) {
```

```

        cout << B[i] - A[i] << " \n"[i == 9];
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);

    int _T = 1;
    while (_T--) solve();
    return 0;
}

```

2.线性 DP

2.1 最长上升子序列 I

状态定义： $f[i]$ 表示以 $a[i]$ 结尾的最长严格上升子序列长度。

状态转移： 枚举前面的 j ，如果 $a[j] < a[i]$ ，那么可以从 j 转移到 i ：

$$f[i] = \max(f[i], f[j] + 1)$$

时间复杂度： $O(n^2)$

```

#include<bits/stdc++.h>

using namespace std;
using ll = long long;

const int N = 1010;

int a[N], f[N];

void solve() {
    int n;
    cin >> n;

    for (int i = 1; i <= n; i++) {
        cin >> a[i];
    }

    int res = 0;
    for (int i = 1; i <= n; i++) {
        f[i] = 1; // 只选自己
        for (int j = 1; j < i; j++) {
            if (a[j] < a[i]) {
                f[i] = max(f[i], f[j] + 1);
            }
        }
        res = max(res, f[i]);
    }
}

```

```
    cout << res << "\n";  
}
```

2.2 最长上升子序列 II

定义： `q[len]` 表示长度为 `len + 1` 的严格上升子序列的**最小结尾值**。

做法： 对于每个 `a[i]`：

- 在 `q` 中找到第一个 `>= a[i]` 的位置
- 用 `a[i]` 去替换它
- 如果没找到，说明 `a[i]` 比当前所有结尾都大，可以直接接到后面

这样最后 `q` 的长度就是 LIS 长度。

```
#include<bits/stdc++.h>  
  
using namespace std;  
using ll = long long;  
  
const int N = 1e5 + 10;  
  
int a[N];  
  
void solve() {  
    int n;  
    cin >> n;  
  
    for (int i = 1; i <= n; i++) cin >> a[i];  
  
    vector<int> q;  
  
    for (int i = 1; i <= n; i++) {  
        auto it = lower_bound(q.begin(), q.end(), a[i]);  
        if (it == q.end()) q.push_back(a[i]);  
        else *it = a[i];  
    }  
  
    cout << q.size() << "\n";  
}
```

二、数据结构

1.前缀与差分类

1.1 前缀置换

用途：维护前缀交换操作形成的排列，查询区间操作后的最终排列。

复杂度：预处理 $O(10n)$ ，单次查询 $O(10 \log 10)$ 。

注意：这里只适合值域固定很小（这里是 10 个球）的情况。

```
#include <bits/stdc++.h>
using namespace std;

void solve() {
    int n, m;
    cin >> n >> m;
    vector<int> cur(10);
    iota(cur.begin(), cur.end(), 0);

    vector<vector<int>> pre;
    pre.push_back(cur);
    for (int i = 1; i <= n; i++) {
        int l, r;
        cin >> l >> r;
        swap(cur[l], cur[r]);
        pre.push_back(cur);
    }

    while (m--) {
        int l, r;
        cin >> l >> r;
        vector<int> p(10);
        iota(p.begin(), p.end(), 0);
        sort(p.begin(), p.end(), [&](const int &lhs, const int &rhs) {
            return pre[l - 1][lhs] < pre[l - 1][rhs];
        });
        for (auto it : pre[r]) {
            cout << p[it] << " \n"[it == pre[r].back()];
        }
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);
    solve();
    return 0;
}
```

1.2 前缀矩阵积

用途：2×2 转移矩阵区间连乘。

复杂度：预处理 $O(n)$ ，单次查询 $O(1)$ 次矩阵运算。

注意：这里用到了矩阵求逆来实现区间矩阵积：

$$M_{[l,r]} = pre_{l-1}^{-1} \cdot pre_r$$

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;

const int MAX_MAT = 2;
const ll mod = 1e9 + 7;

struct Mat {
    ll a[MAX_MAT][MAX_MAT];
    Mat() {
        for (int i = 0; i < MAX_MAT; i++) {
            for (int j = 0; j < MAX_MAT; j++) a[i][j] = 0;
            a[i][i] = 1;
        }
    }
    Mat(ll a1, ll a2, ll a3, ll a4) {
        a[0][0] = a1; a[0][1] = a2;
        a[1][0] = a3; a[1][1] = a4;
    }
};

ll qmi(ll a, ll b, ll MOD = 9223372036854775807LL) {
    ll res = 1;
    while (b) {
        if (b & 1) res = res * a % MOD;
        a = a * a % MOD;
        b >>= 1LL;
    }
    return res;
}

ll A[MAX_MAT][MAX_MAT << 1];
ll get_inv(ll x) { return qmi(x, mod - 2, mod); }

void row_minus(int a, int b, long long k) {
    for (int i = 0; i < 2 * MAX_MAT; ++i) {
        A[a][i] = (A[a][i] - A[b][i] * k % mod) % mod;
        if (A[a][i] < 0) A[a][i] += mod;
    }
}

void row_multiplies(int a, long long k) {
    for (int i = 0; i < 2 * MAX_MAT; ++i) A[a][i] = A[a][i] * k % mod;
}
```

```

void row_swap(int a, int b) {
    for (int i = 0; i < 2 * MAX_MAT; ++i) swap(A[a][i], A[b][i]);
}

Mat getinv(Mat x) {
    memset(A, 0, sizeof(A));
    for (int i = 0; i < MAX_MAT; ++i) {
        for (int j = 0; j < MAX_MAT; ++j) {
            A[i][j] = x.a[i][j];
            A[i][MAX_MAT + j] = (i == j);
        }
    }
    for (int i = 0; i < MAX_MAT; ++i) {
        if (!A[i][i]) {
            for (int j = i + 1; j < MAX_MAT; ++j) {
                if (A[j][i]) {
                    row_swap(i, j);
                    break;
                }
            }
        }
        row_multiplies(i, get_inv(A[i][i]));
        for (int j = i + 1; j < MAX_MAT; ++j) row_minus(j, i, A[j][i]);
    }
    for (int i = MAX_MAT - 1; i >= 0; --i) {
        for (int j = i - 1; j >= 0; --j) row_minus(j, i, A[j][i]);
    }
    Mat ret;
    for (int i = 0; i < MAX_MAT; ++i)
        for (int j = 0; j < MAX_MAT; ++j)
            ret.a[i][j] = A[i][MAX_MAT + j];
    return ret;
}

Mat operator * (Mat x, Mat y) {
    Mat c;
    for (int i = 0; i < MAX_MAT; ++i)
        for (int j = 0; j < MAX_MAT; ++j)
            c.a[i][j] = 0;
    for (int i = 0; i < MAX_MAT; ++i)
        for (int j = 0; j < MAX_MAT; ++j)
            for (int k = 0; k < MAX_MAT; ++k)
                c.a[i][j] = (c.a[i][j] + x.a[i][k] * y.a[k][j] % mod) % mod;
    return c;
}

const int MAXN = 100005;
const Mat tA(1, 1, 0, 1);
const Mat tB(1, 0, 1, 1);
Mat presum[MAXN];

void solve() {
    int n, m;

```

```

cin >> n >> m;
string s;
cin >> s;
s = " " + s;
presum[0] = Mat(1, 0, 0, 1);
for (int i = 1; i < n; i++) {
    if (s[i] == '/') presum[i] = presum[i - 1] * tA;
    else presum[i] = presum[i - 1] * tB;
}
while (m--) {
    int hs, ht, ps, pt;
    cin >> hs >> ht >> ps >> pt;
    Mat ans = getinv(presum[hs - 1]) * presum[ht - 1];
    cout << ans.a[ps][pt] << "\n";
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);
    solve();
    return 0;
}

```

1.3 前缀和 + 等差数列

用途：区间贡献是 $1, 2, 3, \dots$ 形式时，用多次前缀和实现。

复杂度： $O(n)$ 。

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;

const ll mod = 1e9 + 7;

void solve() {
    int n;
    cin >> n;
    string s;
    cin >> s;
    s = " " + s;
    vector<ll> a(n + 10, 0);
    for (int i = 1; i <= n; i++) {
        if (s[i] == '1') a[i + 1]++;
    }
    for (int i = 1; i <= n; i++) a[i] += a[i - 1];
    for (int i = 1; i <= n; i++) a[i] += a[i - 1];

    ll ans = 0;
    for (int i = 1; i <= n; i++) {
        if (s[i] == '1') {

```

```

        ans = (ans + a[i]) % mod;
    }
}
cout << ans << "\n";
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);
    solve();
    return 0;
}

```

1.4 前缀和 + 平方数列

用途：区间后缀加 1, 4, 9, ... 这类可由多次前缀和生成的序列。

技巧：

- 常数列：1 次前缀和
- 等差列：2 次前缀和
- 平方列：3 次前缀和

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;

const ll mod = 1e9 + 7;

void solve() {
    int n, m;
    cin >> n >> m;
    vector<ll> d1(n + 10, 0), d2(n + 10, 0), d3(n + 10, 0);
    while (m--) {
        int type, pos;
        cin >> type >> pos;
        if (type == 1) d1[pos]++;
        else if (type == 2) d2[pos]++;
        else {
            d3[pos]++;
            d3[pos + 1]++;
        }
    }

    auto add = [&](vector<ll>& a) {
        for (int i = 1; i <= n; i++) a[i] += a[i - 1];
    };

    add(d1);
    add(d2); add(d2);
    add(d3); add(d3); add(d3);
}

```

```

    for (int i = 1; i <= n; i++) {
        cout << (d1[i] + d2[i] + d3[i]) % mod << " \n"[i == n];
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);
    int _T = 1;
    cin >> _T;
    while (_T--) solve();
    return 0;
}

```

1.5 前缀和 + 多项式

用途：静态批量区间加多项式，最后统一查询区间和。**核心：**高阶差分 + 小次数多项式。**注意：**这份代码是竞赛向写法，适合 $k \leq 5$ 。

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;

const int mod = 1e9 + 7;

ll qmi(ll a, ll b) {
    ll res = 1;
    a %= mod;
    while (b > 0) {
        if (b & 1) res = res * a % mod;
        a = a * a % mod;
        b >>= 1;
    }
    return res;
}

void solve() {
    int n, m, q;
    cin >> n >> m >> q;
    vector<int> a(n + 20, 0);
    for (int i = 1; i <= n; i++) cin >> a[i];

    auto D = [&](vector<int>& b, int len, int cnt) {
        for (int i = 1; i <= cnt; i++) {
            for (int j = len; j >= 1; j--) {
                b[j] -= b[j - 1];
                b[j] = (b[j] + mod) % mod;
            }
        }
    };
};

```

```

auto f = [&](int x, vector<int>& c, int k) {
    int res = 0;
    for (int i = 0; i <= k; i++) {
        res += c[i] * qmi(x, k - i) % mod;
        res %= mod;
    }
    return res;
};

auto P = [&](vector<int>& b, int len, int cnt) {
    for (int i = 1; i <= cnt; i++) {
        for (int j = 1; j <= len; j++) {
            b[j] += b[j - 1];
            b[j] %= mod;
        }
    }
};

D(a, n, 6);

while (m--) {
    int l, r, k;
    cin >> l >> r >> k;
    vector<int> c(11, 0), c1(11, 0), c2(11, 0);
    for (int i = 0; i <= k; i++) cin >> c[i];
    for (int i = 1; i <= 10; i++) {
        c1[i] = f(i, c, k);
        c2[i] = mod - f(i + r - l + 1, c, k);
    }
    D(c1, 10, 6);
    D(c2, 10, 6);
    for (int i = 1; i <= 10; i++) {
        a[l + i - 1] = (a[l + i - 1] + c1[i]) % mod;
        a[r + i] = (a[r + i] + c2[i]) % mod;
    }
}

P(a, n, 7);
while (q--) {
    int l, r;
    cin >> l >> r;
    cout << (a[r] - a[l - 1] + mod) % mod << "\n";
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);
    solve();
    return 0;
}

```

1.6 高阶前缀和 / 差分 (NTT)

用途：对数组做 k 次前缀和 / 差分， k 可正可负且很大。**复杂度：** $O(n \log n)$ 。**核心：**组合数卷积。

```
#include<bits/stdc++.h>
using namespace std;
namespace NTT {
    const long long g = 3;
    const long long p = 998244353;
    long long wn[35];
    long long pow2(long long a, long long b) {
        long long res = 1;
        while (b) {
            if (b & 1) res = res * a % p;
            a = a * a % p;
            b >>= 1;
        }
        return res;
    }
    void getwn() {
        for (int i = 0; i < 25; i++) wn[i] = pow2(g, (p - 1) / (1LL << i));
    }
    void ntt(long long *a, int len, int f) {
        long long i, j = 0, t, k, w, id;
        for (i = 1; i < len - 1; i++) {
            for (t = len; j ^= t >>= 1, ~j & t;);
            if (i < j) swap(a[i], a[j]);
        }
        for (i = 1, id = 1; i < len; i <<= 1, id++) {
            t = i << 1;
            for (j = 0; j < len; j += t) {
                for (k = 0, w = 1; k < i; k++, w = w * wn[id] % p) {
                    long long x = a[j + k], y = w * a[j + k + i] % p;
                    a[j + k] = (x + y) % p;
                    a[j + k + i] = (x - y + p) % p;
                }
            }
        }
        if (f) {
            for (i = 1, j = len - 1; i < j; i++, j--) swap(a[i], a[j]);
            long long inv = pow2(len, p - 2);
            for (i = 0; i < len; i++) a[i] = a[i] * inv % p;
        }
    }
    void mul(long long *a, long long *b, int l1, int l2) {
        int len, i;
        for (len = 1; len <= l1 + l2; len <<= 1);
        for (i = l1 + 1; i <= len; i++) a[i] = 0;
        for (i = l2 + 1; i <= len; i++) b[i] = 0;
        ntt(a, len, 0); ntt(b, len, 0);
        for (i = 0; i < len; i++) a[i] = a[i] * b[i] % p;
        ntt(a, len, 1);
    }
}
```

```

}

const int MAXN = 300005;
const long long mod = 998244353;
int n;
long long a[MAXN], inv[MAXN], ki[MAXN], k;

void init(long long n) {
    inv[0] = inv[1] = 1;
    for (long long i = 2; i <= n; i++) inv[i] = ((mod - mod / i) * inv[mod % i]) % mod;
}

void get_ki(long long k, int len) {
    k = (k % mod + mod) % mod;
    ki[0] = 1;
    for (int i = 1; i < len; ++i) {
        ki[i] = ki[i - 1] * inv[i] % mod * ((k + i - 1) % mod) % mod;
    }
}

int main() {
    NTT::getwn();
    init(100000);
    scanf("%d %lld", &n, &k);
    get_ki(k, n);
    for (int i = 0; i < n; ++i) scanf("%lld", &a[i]);
    NTT::mul(a, ki, n, n);
    for (int i = 0; i < n; ++i) printf("%lld%c", a[i], i == n - 1 ? '\n' : ' ');
    return 0;
}

```

1.7 SOS DP (子集和变换)

用途：查询某集合所有子集 / 超集的值和。 **复杂度：**预处理 $O(n2^n)$, 查询 $O(n)$ 或 $O(1)$ 读答案。

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;

void solve() {
    int n, m;
    cin >> n >> m;
    int maxbit = 1 << n;
    vector<int> a(n, 0);
    for (int i = 0; i < n; i++) cin >> a[i];

    vector<ll> pre(maxbit, 0), suf(maxbit, 0);
    for (int i = 0; i < maxbit; i++) {
        ll sum = 0;
        for (int j = 0; j < n; j++) if (i & (1 << j)) sum ^= a[j];
        pre[i] = suf[i] = sum;
    }
}

```

```

}

for (int i = 0; i < n; i++) {
    for (int j = 0; j < maxbit; j++) {
        if (j & (1 << i)) pre[j] += pre[j ^ (1 << i)];
        else suf[j] += suf[j ^ (1 << i)];
    }
}

while (m--) {
    int k;
    cin >> k;
    int q = 0;
    for (int i = 1; i <= k; i++) {
        int x;
        cin >> x;
        q |= (1 << (x - 1));
    }
    cout << pre[q] << " " << suf[q] << "\n";
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);
    solve();
    return 0;
}

```

2. 区间数据结构

2.1 线段树（区间乘、区间加、区间和、平方和）

用途：同时维护

- 区间和
- 区间平方和
- 区间乘法修改
- 区间加法修改

复杂度：单次修改 / 查询 $O(\log n)$ 。 **注意：**懒标记是仿射变换 $x \mapsto ax + b$ 。

```

#include<bits/stdc++.h>
#define ls fa << 1
#define rs fa << 1 | 1
using namespace std;
using ll = long long;

const int N = 1e5 + 10;
ll a[N];

```

```

struct tnode {
    ll sum[2], lazy[2];
    int l, r;
};

struct Segment_Tree {
    tnode t[4 * N];

    void init_lazy(int fa) {
        t[fa].lazy[0] = 1;
        t[fa].lazy[1] = 0;
    }

    void union_lazy(int fa, int ch) {
        ll temp[2];
        temp[0] = t[fa].lazy[0] * t[ch].lazy[0];
        temp[1] = t[fa].lazy[0] * t[ch].lazy[1] + t[fa].lazy[1];
        t[ch].lazy[0] = temp[0];
        t[ch].lazy[1] = temp[1];
    }

    void cal_lazy(int fa) {
        t[fa].sum[1] = t[fa].lazy[0] * t[fa].lazy[0] * t[fa].sum[1]
            + t[fa].lazy[1] * t[fa].lazy[1] * (t[fa].r - t[fa].l + 1)
            + t[fa].lazy[0] * t[fa].lazy[1] * 2 * t[fa].sum[0];
        t[fa].sum[0] = t[fa].lazy[0] * t[fa].sum[0]
            + t[fa].lazy[1] * (t[fa].r - t[fa].l + 1);
    }

    void push_down(int fa) {
        if (t[fa].lazy[0] != 1 || t[fa].lazy[1] != 0) {
            cal_lazy(fa);
            if (t[fa].l != t[fa].r) {
                union_lazy(fa, ls);
                union_lazy(fa, rs);
            }
            init_lazy(fa);
        }
    }

    void update(int fa) {
        push_down(ls);
        push_down(rs);
        t[fa].sum[0] = t[ls].sum[0] + t[rs].sum[0];
        t[fa].sum[1] = t[ls].sum[1] + t[rs].sum[1];
    }

    void build(int fa, int l, int r) {
        t[fa].l = l;
        t[fa].r = r;
        init_lazy(fa);
        if (l == r) {

```

```

        t[fa].sum[0] = a[l];
        t[fa].sum[1] = a[l] * a[l];
        return;
    }
    ll mid = (l + r) >> 1;
    build(ls, l, mid);
    build(rs, mid + 1, r);
    update(fa);
}

```

```

void change(int fa, int l, int r, ll delta, int op) {
    push_down(fa);
    if (l == t[fa].l && r == t[fa].r) {
        t[fa].lazy[op] = delta;
        return;
    }
    int mid = (t[fa].l + t[fa].r) >> 1;
    if (r <= mid) change(ls, l, r, delta, op);
    else if (l > mid) change(rs, l, r, delta, op);
    else {
        change(ls, l, mid, delta, op);
        change(rs, mid + 1, r, delta, op);
    }
    update(fa);
}

```

```

ll sum(int fa, int l, int r, int op) {
    push_down(fa);
    if (t[fa].l == l && t[fa].r == r) return t[fa].sum[op];
    int mid = (t[fa].l + t[fa].r) >> 1;
    if (r <= mid) return sum(ls, l, r, op);
    else if (l > mid) return sum(rs, l, r, op);
    else return sum(ls, l, mid, op) + sum(rs, mid + 1, r, op);
}
};

```

Segment_Tree ST;

```

void solve() {
    int n, m;
    cin >> n >> m;
    for (int i = 1; i <= n; i++) cin >> a[i];
    ST.build(1, 1, n);
    for (int i = 1; i <= m; i++) {
        ll op, l, r;
        cin >> op >> l >> r;
        if (op >= 3) {
            ll x;
            cin >> x;
            ST.change(1, l, r, x, op - 3);
        } else {
            cout << ST.sum(1, l, r, op - 1) << "\n";
        }
    }
}

```

```
}  
}
```

2.2 树状数组 1：单点加 / 区间和

复杂度：单次修改 / 查询 $O(\log n)$ 。

```
#include<bits/stdc++.h>  
using namespace std;  
  
const int N = 5e5 + 10;  
int n, m, tr[N];  
  
int lowbit(int x) { return x & -x; }  
  
void change(int x, int k) {  
    while (x <= n) tr[x] += k, x += lowbit(x);  
}  
  
int query(int x) {  
    int t = 0;  
    while (x) t += tr[x], x -= lowbit(x);  
    return t;  
}  
  
void solve() {  
    cin >> n >> m;  
    for (int i = 1; i <= n; i++) {  
        int y;  
        cin >> y;  
        change(i, y);  
    }  
    for (int i = 1; i <= m; i++) {  
        int op, x, y;  
        cin >> op >> x >> y;  
        if (op == 1) change(x, y);  
        else cout << query(y) - query(x - 1) << "\n";  
    }  
}
```

2.3 树状数组 2：区间加 / 单点查

技巧：维护差分数组的前缀和。 **复杂度：**单次 $O(\log n)$ 。

```
#include<bits/stdc++.h>  
using namespace std;  
  
const int N = 5e5 + 10;  
int n, m, a[N];
```

```

int lowbit(int x) { return x & -x; }

void change(int x, int k) {
    while (x <= n) a[x] += k, x += lowbit(x);
}

int query(int x) {
    int t = 0;
    while (x) t += a[x], x -= lowbit(x);
    return t;
}

void solve() {
    cin >> n >> m;
    for (int i = 1; i <= n; i++) {
        int y;
        cin >> y;
        change(i, y);
        change(i + 1, -y);
    }
    for (int i = 1; i <= m; i++) {
        int op;
        cin >> op;
        if (op == 1) {
            int x, y, k;
            cin >> x >> y >> k;
            change(x, k);
            change(y + 1, -k);
        } else {
            int x;
            cin >> x;
            cout << query(x) << "\n";
        }
    }
}

```

2.4 树状数组处理二维偏序

用途：把二维条件转成排序 + 一维树状数组。 **套路：**

- 对第一维“排序”
- 对第二维“离散化”
- 用“树状数组”维护第二维

注意：

严格 >：同 u 时， v 降序 **非严格 $\geq$ ：**同 u 时， v 升序

```

/*
给定 n 个出现时间为 y、位置为 x 的物品
求从原点出发、最大移动速度为 1 的情况下，最多能按序拾取多少个物品

```

```

*/
#include<bits/stdc++.h>

#define int long long

using namespace std;
using pii = pair<int, int>;

const int N = 2e3 + 10;

int tr[N];

int lowbit(int x) { return x & (-x); }

void change(int x, int k) {
    for (int i = x; i < N; i += lowbit(i)) tr[i] = max(tr[i], k);
}

int query(int x) {
    int res = 0;
    while (x) {
        res = max(res, tr[x]);
        x -= lowbit(x);
    }
    return res;
}

void solve() {
    int n;
    cin >> n;
    vector<int> alls;
    vector<pii> p;
    for (int i = 1; i <= n; i++) {
        int x, y;
        cin >> x >> y;
        if (y >= x) {
            int u = y - x, v = y + x;
            alls.push_back(v);
            p.push_back({u, v});
        }
    }
    if (p.empty()) {
        cout << 0 << "\n";
        return;
    }
    sort(alls.begin(), alls.end());
    alls.erase(unique(alls.begin(), alls.end()), alls.end());
    sort(p.begin(), p.end());
    int ans = 0;
    for (auto [u, v] : p) {
        int it = lower_bound(alls.begin(), alls.end(), v) - alls.begin() + 1;
        int cur = query(it) + 1;
        ans = max(ans, cur);
    }
}

```

```

        change(it, cur);
    }
    cout << ans << "\n";
}

//int main() {
signed main() {
    ios::sync_with_stdio(false);
    cin.tie(0);
    int _T = 1;
    // cin >> _T;
    while (_T--) solve();
    return 0;
}

```

3.倍增与 RMQ

3.1 ST 表

用途：静态 RMQ。 **复杂度：**预处理 $O(n \log n)$ ，查询 $O(1)$ 。 **注意：**适合静态问题，不支持修改。

```

void solve() {
    int n, m;
    cin >> n >> m;
    vector<int> a(n + 1, 0);
    for (int i = 1; i <= n; i++) cin >> a[i];

    int x = floor(log2(n));
    vector<vector<int>> dp(n + 1, vector<int>(x + 1, 0));

    for (int j = 0; j <= x; j++) {
        int len = 1 << j;
        for (int i = 1; i + len - 1 <= n; i++) {
            if (j == 0) dp[i][j] = a[i];
            else dp[i][j] = max(dp[i][j - 1], dp[i + (len >> 1)][j - 1]);
        }
    }

    while (m--) {
        int l, r;
        cin >> l >> r;
        int y = floor(log2(r - l + 1));
        cout << max(dp[l][y], dp[r - (1 << y) + 1][y]) << "\n";
    }
}

```

3.2 倍增表

用途：每点恰有一条出边时，求走 k 步后的信息。 **可维护：**

- 第 2^j 步到哪
- 权值和
- 权值最小值

```
/*
每个点出发经过 k 次跳转后落到哪个点，并输出那个点的权值。
*/

int to[N][60];
/*
to数组建议写全局

或者这样写,常数要小一点
vector<array<int, 60>> to(n + 1);
*/

void solve() {
    ll n, k;
    cin >> n >> k;
    vector<int> a(n + 1, 0), nex(n + 1, 0);
    for (int i = 1; i <= n; i++) {
        cin >> nex[i];
    }
    for (int i = 1; i <= n; i++) {
        cin >> a[i];
    }
    for (int i = 1; i <= n; i++) {
        to[i][0] = nex[i];
    }
    for (int j = 1; j <= 59; j++) {
        for (int i = 1; i <= n; i++) {
            int st = to[i][j - 1];
            to[i][j] = to[st][j - 1];
        }
    }
    for (int i = 1; i <= n; i++) {
        int p = i;
        for (ll j = 59; j >= 0; j--) {
            if ((k >> j) & 1LL) {
                p = to[p][j]; // p的位置记得更新
            }
        }
        cout << a[p] << " ";
    }
}
```

4.其他常用结构

4.1 三维前缀和

复杂度：预处理 $O(n^3)$ ，单次查询 $O(1)$ 。

```
const int N = 110;
int f[N][N][N], g[N][N][N];

void solve() {
    int n;
    cin >> n;
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= n; j++)
            for (int k = 1; k <= n; k++)
                cin >> f[i][j][k];

    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= n; j++)
            for (int k = 1; k <= n; k++)
                g[i][j][k] = g[i][j][k - 1] + g[i][j - 1][k] + g[i - 1][j][k]
                    - g[i - 1][j - 1][k] - g[i - 1][j][k - 1] - g[i][j - 1][k - 1]
                    + g[i - 1][j - 1][k - 1] + f[i][j][k];

    int q;
    cin >> q;
    while (q--) {
        int x, y, z, xx, yy, zz;
        cin >> x >> xx >> y >> yy >> z >> zz;
        int res = g[xx][yy][zz] - g[xx][yy][z - 1] - g[xx][y - 1][zz] - g[x - 1][yy][zz]
            + g[x - 1][y - 1][zz] + g[x - 1][yy][z - 1] + g[xx][y - 1][z - 1]
            - g[x - 1][y - 1][z - 1];
        cout << res << '\n';
    }
}
```

4.2 字符串哈希（双模）

用途：判断两个子串是否相等。 **复杂度：**预处理 $O(n)$ ，查询 $O(1)$ 。 **注意：**双模更稳。

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;

const int N = 1e5 + 10;
const array<ll, 2> mod = {1000000007, 998244353};
const array<ll, 2> P = {131, 137};
array<ll, 2> p[N], h[N];
string s;

array<ll, 2> get_hash(int l, int r) {
```

```

array<ll, 2> res;
for (int k = 0; k < 2; k++) {
    res[k] = h[r][k] - h[l - 1][k] * p[r - l + 1][k] % mod[k];
    res[k] = (res[k] % mod[k] + mod[k]) % mod[k];
}
return res;
}

void solve() {
    int n, m;
    cin >> n >> m;
    cin >> s;
    s = " " + s;

    for (int i = 0; i < 2; i++) {
        p[0][i] = 1;
        h[0][i] = 0;
    }
    for (int k = 0; k < 2; k++) {
        for (int i = 1; i <= n; i++) {
            p[i][k] = p[i - 1][k] * P[k] % mod[k];
            h[i][k] = (h[i - 1][k] * P[k] + s[i]) % mod[k];
        }
    }

    while (m--) {
        int l1, r1, l2, r2;
        cin >> l1 >> r1 >> l2 >> r2;
        cout << (get_hash(l1, r1) == get_hash(l2, r2) ? "Yes\n" : "No\n");
    }
}

```

4.3 最小表示法

用途：求循环串中字典序最小的表示。 **复杂度：** $O(n)$ 。

```

void solve() {
    int n;
    cin >> n;
    string s;
    cin >> s;
    s = " " + s + s;

    int i = 1, j = 2;
    while (i <= n && j <= n) {
        int k = 0;
        while (k < n && s[i + k] == s[j + k]) k++;
        if (k == n) break;
        if (s[i + k] > s[j + k]) i = i + k + 1;
        else j = j + k + 1;
        if (i == j) j++;
    }
}

```

```

}

int t = min(i, j);
for (int i = t; i <= n; i++) cout << s[i];
for (int i = 1; i < t; i++) cout << s[i];
}

```

4.4 rope

用途：大字符串 / 大序列的拼接、插入、删除。**复杂度：**一般视作平衡树，单次操作近似 $O(\log n)$ 。**注意：**GCC 扩展。

```

#include<bits/stdc++.h>
#include <ext/rope>
using namespace std;
using namespace __gnu_cxx;

void solve() {
    int n, q;
    cin >> n >> q;
    rope<char> server;
    vector<rope<char>> pcs(n + 1);

    while (q--) {
        int type, p;
        cin >> type >> p;
        if (type == 1) pcs[p] = server;
        else if (type == 2) {
            string s;
            cin >> s;
            pcs[p].insert(pcs[p].size(), s.c_str(), s.size());
        } else {
            server = pcs[p];
        }
    }

    for (auto it = server.begin(); it != server.end(); it++) cout << *it;
    cout << '\n';
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);
    solve();
    return 0;
}

```

4.5 ordered_set

用途：支持求排名、求第 k 小。**注意：**不能直接存重复值，若要存重复值可存 `pair<值, 编号>`。

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;

template<typename T>
using ordered_set = tree<T, null_type, less<T>,
                        rb_tree_tag,
                        tree_order_statistics_node_update>;
```

```
/*
order_of_key(x) 返回严格小于 x 的元素个数
find_by_order(k) 返回指向第 k 个元素的迭代器 (0-based)
*/

ordered_set<pair<int, int>> st;
int cnt = 0;
st.insert({x, ++cnt});
```

4.6 普通可换根 Trie (统计前缀数量版)

用途：

- 插入字符串
- 查询某前缀出现次数
- 整体加同一前缀字符 (换根)

```
#include<bits/stdc++.h>
using namespace std;

const int N = 2e6 + 10;
const int SIGMA = 62;

int to[N][SIGMA], cnt[N], idx;
int root, tot;

int id(char x) {
    if (x >= 'A' && x <= 'Z') return x - 'A';
    if (x >= 'a' && x <= 'z') return x - 'a' + 26;
    return x - '0' + 52;
}

void init() {
    for (int i = 0; i <= idx; i++) {
        cnt[i] = 0;
        for (int j = 0; j < SIGMA; j++) to[i][j] = 0;
    }
}
```

```

    idx = 1;
    root = 1;
    tot = 0;
}

void add(char ch) {
    if (tot == 0) return;
    int u = id(ch);
    ++idx;
    to[idx][u] = root;
    cnt[idx] = cnt[root];
    root = idx;
}

void insert(const string &s) {
    int p = root;
    tot++;
    cnt[p]++;
    for (char c : s) {
        int u = id(c);
        if (!to[p][u]) to[p][u] = ++idx;
        p = to[p][u];
        cnt[p]++;
    }
}

int query(const string &s) {
    int p = root;
    for (char c : s) {
        int u = id(c);
        if (!to[p][u]) return 0;
        p = to[p][u];
    }
    return cnt[p];
}

```

三、图论

1. 树的重心

定义

删除某个点后，剩余各个连通块的点数最大值最小，则这个点就是树的重心。

思路

设 `sum` 表示当前点 `u` 的子树大小。

删除 `u` 后，会形成若干连通块：

- 每棵子树各自形成一个连通块，大小为对应子树大小

- 除了 `u` 子树以外的其余部分也形成一个连通块，大小为 `n - sum`

取这些块大小的最大值，记为 `mx`。所有点里 `mx` 最小的那个点就是重心。

模板

```
const int N = 1e5 + 10;
vector<int> es[N];
int n;
int ans = N;

int dfs(int u, int fa) {
    int sum = 1;
    int mx = 0;

    for (auto v : es[u]) {
        if (v == fa) continue;
        int t = dfs(v, u);
        mx = max(mx, t);
        sum += t;
    }

    mx = max(mx, n - sum);
    ans = min(ans, mx);

    return sum;
}
```

2. Bellman-Ford

有边数限制的最短路

适用场景

- 图中可能有负边
- 限制最多经过 `k` 条边
- 求最短路

核心思想

进行 `k` 轮松弛。每一轮表示允许再多走一条边。

这里一定要开 `backup[]` 备份上一轮的结果。否则一轮中更新出来的新值又被继续拿去转移，就相当于一轮走了多条边。

模板

```
const int N = 10010;
const int M = 510;
const int INF = 0x3f3f3f3f;

int a[N], b[N], c[N];
int dist[M], backup[M];
```

```

int n, m, k;

int bellman_ford() {
    memset(dist, 0x3f, sizeof dist);
    dist[1] = 0;

    for (int i = 0; i < k; i++) {
        memcpy(backup, dist, sizeof dist);
        for (int j = 0; j < m; j++) {
            dist[b[j]] = min(dist[b[j]], backup[a[j]] + c[j]);
        }
    }

    return dist[n];
}

```

注意点

- 一定要用 `backup`
- 判断无解常用 `res > INF / 2`
- 即使图里有负环，只要限制边数，这题仍然能做

3. SPFA

3.1 SPFA 求最短路

适用场景

- 图中有负边
- 没有负环
- 求单源最短路

核心思想

SPFA 是 Bellman-Ford 的队列优化。

只有当前被更新过的点，才继续拿去松弛它连出去的边。

模板

```

const int N = 1e5 + 10;
const int INF = 0x3f3f3f3f;
vector<pair<int, int>> es[N];
int dist[N];
bool st[N];
int n, m;

int spfa() {
    memset(dist, 0x3f, sizeof dist);
    queue<int> q;

    dist[1] = 0;
}

```

```

q.push(1);
st[1] = true;

while (!q.empty()) {
    int u = q.front();
    q.pop();
    st[u] = false;

    for (auto [v, w] : es[u]) {
        if (dist[v] > dist[u] + w) {
            dist[v] = dist[u] + w;
            if (!st[v]) {
                q.push(v);
                st[v] = true;
            }
        }
    }
}

return dist[n];
}

```

注意点

- `st[v]` 表示点 `v` 是否在队列中
- 它不表示这个点已经确定最短路
- SPFA 最坏复杂度仍然是 $O(nm)$

3.2 SPFA 判断负环

核心思想

如果某个点在最短路更新过程中，经过的边数 `cnt[v] >= n`，说明图中存在负环。

因为一个不重复经过点的简单路径最多只会经过 `n - 1` 条边。

如果还能继续更新，就说明绕了负环。

模板

```

const int N = 2010;
vector<pair<int, int>> es[N];
int dist[N], cnt[N];
bool st[N];
int n, m;

bool spfa() {
    queue<int> q;
    for (int i = 1; i <= n; i++) {
        q.push(i);
        st[i] = true;
    }
}

```

```

while (!q.empty()) {
    int u = q.front();
    q.pop();
    st[u] = false;

    for (auto [v, w] : es[u]) {
        if (dist[v] > dist[u] + w) {
            dist[v] = dist[u] + w;
            cnt[v] = cnt[u] + 1;
            if (cnt[v] >= n) return true;

            if (!st[v]) {
                q.push(v);
                st[v] = true;
            }
        }
    }
}

return false;
}

```

注意点

- 要把所有点一开始都入队
- 因为负环不一定从 1 号点可达
- 这是在判断整张图是否有负环

4. Floyd

4.1 Floyd 求最短路

适用场景

- 多源最短路
- 询问很多次任意两点最短路
- 点数较小，一般 $n \leq 200$

核心思想

设 $d[i][j]$ 表示 $i \rightarrow j$ 的最短距离。

枚举中转点 k ，尝试更新：

```
d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
```

本质上是一个动态规划。

核心思想

```
const int N = 210;
```

```

const int INF = 0x3f3f3f3f;
int d[N][N];

int main() {
    int n, m, q;
    cin >> n >> m >> q;

    memset(d, 0x3f, sizeof d);
    for (int i = 1; i <= n; i++) d[i][i] = 0;

    while (m--) {
        int a, b, c;
        cin >> a >> b >> c;
        d[a][b] = min(d[a][b], c);
    }

    for (int k = 1; k <= n; k++) {
        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= n; j++) {
                d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
            }
        }
    }

    while (q--) {
        int a, b;
        cin >> a >> b;
        if (d[a][b] > INF / 2) cout << "impossible\n";
        else cout << d[a][b] << "\n";
    }

    return 0;
}

```

注意点

- 初始化时 `d[i][i] = 0`
- 有重边要取最小值
- 判断无路常用 `INF / 2`

5. 二分图

5.1 染色法判断二分图

定义

如果能把图中所有点染成两种颜色，并且每条边的两个端点颜色都不同，那么这个图就是二分图。

等价结论

一个图是二分图，当且仅当它不含奇环。

DFS 染色模板

```
const int N = 1e5 + 10;
vector<int> es[N];
int color[N];

bool dfs(int u, int c) {
    color[u] = c;
    for (auto v : es[u]) {
        if (!color[v]) {
            if (!dfs(v, 3 - c)) return false;
        } else if (color[v] == c) {
            return false;
        }
    }
    return true;
}
```

主程序:

```
bool ok = true;
for (int i = 1; i <= n; i++) {
    if (color[i]) continue;
    if (!dfs(i, 1)) {
        ok = false;
        break;
    }
}
cout << (ok ? "Yes" : "No");
```

BFS 染色模板

```
bool bfs(int s) {
    queue<int> q;
    q.push(s);
    color[s] = 1;

    while (!q.empty()) {
        int u = q.front();
        q.pop();

        for (auto v : es[u]) {
            if (!color[v]) {
                color[v] = 3 - color[u];
                q.push(v);
            } else if (color[v] == color[u]) {
                return false;
            }
        }
    }

    return true;
}
```

```
}
```

5.2 二分图的最大匹配

定义

匹配指的是若干条边，任意两条边都没有公共端点。
边数最多的匹配叫最大匹配。

常用做法

匈牙利算法。

核心思想

依次枚举左部点，看它能不能找到一条增广路。
如果能找到，就让匹配数加一。

邻接表模板

```
const int N = 510;
vector<int> es[N];
int match[N];
bool st[N];

bool find(int u) {
    for (auto v : es[u]) {
        if (st[v]) continue;
        st[v] = true;

        if (!match[v] || find(match[v])) {
            match[v] = u;
            return true;
        }
    }
    return false;
}
```

主程序：

```
int res = 0;
for (int i = 1; i <= n1; i++) {
    memset(st, false, sizeof st);
    if (find(i)) res++;
}
cout << res;
```

理解重点

如果右部点 v 已经匹配给别人了，那么尝试看看原来那个左部点能不能换到别的右部点。
如果能换开，就把 v 让给当前点，这样匹配数就增加了。

这就是增广路的本质。

6. 传递闭包

适用场景

求有向图中任意两点是否可达。

复杂度

- Floyd: $O(n^3)$
- bitset 优化: $O(\frac{n^3}{w})$, 竞赛里通常可看成 $O(n^3/32)$ 或 $O(n^3/64)$

注意点

- 输入是邻接矩阵时最好直接做。
- bitset 写法里, 外层枚举中转点, 内层枚举行点, 且判断 `f[i][j]` 后做按位或。

Floyd 版

```
#include<bits/stdc++.h>

using namespace std;

//传统Floyd 算法
void solve() {
    int n;
    cin >> n;
    vector<vector<int>>> f(n + 10, vector<int>(n + 10, 0));
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            cin >> f[i][j];
        }
    }
    for (int k = 1; k <= n; k++) {
        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= n; j++) {
                f[i][j] |= f[i][k] & f[k][j];
            }
        }
    }
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            cout << f[i][j] << ' ';
        }
        cout << '\n';
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);
    int _T = 1;
    // cin >> _T;
    while (_T--) solve();
    return 0;
}
```

```
}
```

bitset 优化版

```
#include<bits/stdc++.h>

using namespace std;

const int N = 100 + 10;
const int M = 256;

bitset<M> f[N]; //基于Floyd-Warshall算法 M和列数有关
//时间复杂度为 $O(n^3/32)$ 
void solve() {
    int n;
    cin >> n;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            int x;
            cin >> x; //通过中间变量来实现输入
            f[i][j] = x;
        }
    }
    for (int j = 1; j <= n; j++) { //注意j循环在i循环外
        for (int i = 1; i <= n; i++) {
            if (f[i][j]) f[i] |= f[j];
        }
    }
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            cout << f[i][j] << ' ';
        }
        cout << "\n";
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);
    int _T = 1;
    // cin >> _T;
    while (_T--) solve();
    return 0;
}
```

7. 树的直径

7.1 核心结论

- 树上任意两点距离的最大值称为树的直径。
- 从任意点出发，走到的最远点一定是某条直径的一个端点。
- 求出直径端点 U, V 后，任意点 x 的最远距离为：

$$\max(\text{dist}(x, U), \text{dist}(x, V))$$

7.2 两次 DFS / BFS

适用场景

边权非负，要求直径长度，或者还想顺手找直径端点。

复杂度

$O(n)$

注意点

- 如果边权可能为负，不要用这个结论直接做。
- 第一次找一个端点，第二次从该端点再跑一次找另一端点。

```
void solve() {
    int n;
    cin >> n;

    vector<vector<int>> es(n + 10);
    for (int i = 1; i < n; i++) {
        int u, v;
        cin >> u >> v;
        es[u].push_back(v);
        es[v].push_back(u);
    }

    int st = -1;
    int mx = -1;

    auto dfs = [&](auto && self, int x, int fa, int d) -> void {
        if (d > mx) {
            mx = d;
            st = x;
        }
        for (auto it : es[x]) {
            if (it == fa) continue;
            self(self, it, x, d + 1);
        }
    };

    dfs(dfs, 1, 0, 0);
}
```

```

    dfs(dfs, st, 0, 0);

    cout << mx << "\n";
}

```

7.3 树形 DP 求直径

适用场景

- 边权可能为负
- 需要嵌入树形 DP
- 只关心直径长度

复杂度

$O(n)$

思路

设 d_1, d_2 表示当前点向下走的最长链和次长链，则经过当前点的最优答案为 $d_1 + d_2$ 。

```

void solve() {
    int n;
    cin >> n;

    vector<vector<int>> es(n + 10);
    for (int i = 1; i < n; i++) {
        int u, v;
        cin >> u >> v;
        es[u].push_back(v);
        es[v].push_back(u);
    }

    int ans = 0;

    auto dfs = [&](auto && self, int u, int fa) -> int {
        int d1 = 0, d2 = 0;
        for (auto it : es[u]) {
            if (it == fa) continue;
            int d = self(self, it, u) + 1;
            if (d > d1) {
                d2 = d1;
                d1 = d;
            } else if (d > d2) {
                d2 = d;
            }
        }

        ans = max(ans, d1 + d2);

        return d1;
    };
}

```

```

dfs(dfs, 1, -1);

cout << ans << "\n";
}

```

7.4 树的中心

树的中心一定在直径中点处。

- 直径长度为偶数：中心有 1 个
- 直径长度为奇数：中心有 2 个

7.5 两棵树合并后的最小直径

若两棵树直径分别为 D_A, D_B ，把两棵树的中心连起来最优。新直径为：

$$D_{new} = \max \left(D_A, D_B, \left\lceil \frac{D_A}{2} \right\rceil + \left\lceil \frac{D_B}{2} \right\rceil + 1 \right)$$

8. 最近公共祖先 LCA

8.1 倍增

适用场景

- 在线回答多次 LCA 询问
- 也常配合树上距离、跳祖先、第 k 级祖先使用

复杂度

- 预处理： $O(n \log n)$
- 单次询问： $O(\log n)$

注意点

- `to[u][i]` 表示 `u` 的 2^i 级祖先。
- 先把深度拉平，再一起往上跳。
- `LOG` 要根据数据范围开够。

```

// 倍增法 O(nlogn)
#include<bits/stdc++.h>

using namespace std;

const int N = 5e5 + 10;

vector<int> es[N];

void solve() {
    int n, m, s;
    cin >> n >> m >> s;
}

```

```

vector<int> dep(n + 10, 0);
vector<vector<int>> to(n + 10, vector<int> (22, 0));

for (int i = 1; i < n; i++) {
    int u, v;
    cin >> u >> v;
    es[u].push_back(v);
    es[v].push_back(u);
}

auto dfs = [&](auto && self, int u, int fa) -> void {
    to[u][0] = fa, dep[u] = dep[fa] + 1;
    for (int i = 1; i <= 20; i++) {
        to[u][i] = to[to[u][i - 1]][i - 1];
    }
    for (auto it : es[u]) {
        if (it == fa) continue;
        self(self, it, u);
    }
};

dfs(dfs, s, 0);

auto lca = [&](int u, int v) -> int {
    if (dep[u] < dep[v]) swap(u, v);
    int deep = dep[u] - dep[v];
    for (int i = 21; i >= 0; i--) {
        if ((deep >> i) & 1) {
            u = to[u][i];
        }
    }
    if (u == v) return v;
    for (int i = 21; i >= 0; i--) {
        if (to[u][i] != to[v][i]) {
            u = to[u][i], v = to[v][i];
        }
    }
    return to[u][0];
};

while (m--) {
    int u, v;
    cin >> u >> v;
    cout << lca(u, v) << "\n";
}
}

```

8.2 Tarjan 离线 LCA

适用场景

所有询问提前给出，整题只离线处理一次。

复杂度

$$O(n + m)$$

注意点

- 本质是 DFS + 并查集。
- 只适合离线，不适合边读边答。

```
// Tarjan算法 O(n+m)
#include<bits/stdc++.h>

using namespace std;
using pii = pair<int, int>;

const int N = 5e5 + 10;

vector<int> es[N];
vector<pii> query[N];
int vis[N], p[N], ans[N];

int find(int x) {
    if (p[x] != x) return p[x] = find(p[x]);
    return p[x];
};

void tarjan(int x) {
    vis[x] = 1;
    for (auto it : es[x]) {
        if (!vis[it]) {
            tarjan(it);
            p[it] = x;
        }
    }
    for (auto [z, y] : query[x]) {
        if (vis[z]) ans[y] = find(z);
    }
}

void solve() {
    int n, m, s;
    cin >> n >> m >> s;

    for (int i = 1; i <= n; i++) {
        p[i] = i;
    }

    for (int i = 1; i < n; i++) {
```

```

        int u, v;
        cin >> u >> v;
        es[u].push_back(v);
        es[v].push_back(u);
    }
    for (int i = 1; i <= m; i++) {
        int u, v;
        cin >> u >> v;
        query[u].push_back({v, i});
        query[v].push_back({u, i});
    }

    tarjan(s);

    for (int i = 1; i <= m; i++) {
        cout << ans[i] << "\n";
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);
    int _T = 1;
    // cin >> _T;
    while (_T--) solve();
    return 0;
}

```

9. 最小生成树

9.1 Kruskal

```

/*
O(m * logm)
每次把最短的边组成一棵树
用并查集维护这些树
若合并次数小于n-1
说明不存在最小生成树
*/

struct Edge {
    int u, v, w;
    bool operator<(const Edge& t) const {
        return w < t.w;
    }
};

void solve() {
    int n, m;
    cin >> n >> m;
}

```

```

vector<Edge> e(m + 1);
for (int i = 1; i <= m; i++) {
    cin >> e[i].u >> e[i].v >> e[i].w;
}

sort(e.begin() + 1, e.begin() + 1 + m);

vector<int> f(n + 1);
iota(f.begin(), f.end(), 0);

auto find = [&](auto && self, int x) -> int {
    if (f[x] == x) return x;
    return f[x] = self(self, f[x]);
};

ll ans = 0;
int cnt = 0;

for (int i = 1; i <= m; i++) {
    int u = e[i].u, v = e[i].v, w = e[i].w;
    int fu = find(find, u), fv = find(find, v);

    if (fu == fv) continue;

    f[fu] = fv;
    ans += w;
    cnt++;

    if (cnt == n - 1) break;
}

if (cnt == n - 1) cout << ans << "\n";
else cout << "orz\n";
}

```

9.2 Prim

```

/*
O(n^2)
Prim: 每次选一个距离当前生成树最近的点加入
d[i] 表示点 i 到生成树的最小距离
若某次最小距离为 inf, 则图不连通
*/

int prim() {
    vector<int> d(n + 1, inf);
    vector<int> st(n + 1, 0);

    d[1] = 0;
    int res = 0;

```

```

for (int i = 1; i <= n; i++) {
    int t = -1;
    for (int j = 1; j <= n; j++) {
        if (!st[j] && (t == -1 || d[j] < d[t])) {
            t = j;
        }
    }

    if (d[t] == inf) return inf;

    st[t] = 1;
    res += d[t];

    for (int j = 1; j <= n; j++) {
        d[j] = min(d[j], g[t][j]);
    }
}

return res;
}

```

9.3 Boruvka

适用场景

- 每轮都能较快找到每个连通块向外的最小边
- 适合一些特殊图、隐式图、补图、完全图变形

复杂度特征

每轮连通块数量至少大幅减少，因此轮数最多 $O(\log n)$ 。总复杂度取决于“每轮如何找每个块的最小出边”。

核心思想

开始时每个点单独成块。

每轮对每个连通块找一条最小出边，把这些边全部加入，然后合并连通块，直到只剩一个连通块。

这份模板的用途

这段代码是针对“边权为 $a_u + a_v$ ，但删去了一些边”的特殊图写法。

```

#include<bits/stdc++.h>
#define int long long

using namespace std;
using ll = long long;
using pii = pair<int, int>;

const ll INF = 0x3f3f3f3f3f3f3f;

void solve() {
    int n, m;
    cin >> n >> m;

```

```

vector<int> a(n + 10, 0), f(n + 10, 0), sz(n + 10, 1);
iota(f.begin() + 1, f.begin() + 1 + n, 1);
for (int i = 1; i <= n; i++) {
    cin >> a[i];
}
vector<vector<int>> es(n + 10);
for (int i = 1; i <= m; i++) {
    int u, v;
    cin >> u >> v;
    es[u].push_back(v);
    es[v].push_back(u);
}

auto find = [&](auto && self, int x) -> int {
    if (f[x] == x) return f[x];
    return f[x] = self(self, f[x]);
};

int ans = 0;
set<pii> se;
for (int i = 1; i <= n; i++) {
    se.insert({a[i], i});
}

auto boruvka = [&]() {
    vector<vector<int>> id(n + 10);
    for (int i = 1; i <= n; i++) {
        id[find(find, i)].push_back(i);
    }
    int cnt = 0;
    for (int i = 1; i <= n; i++) {
        for (auto x : id[i]) {
            se.erase({a[x], x});
        }
        int mi = INF, p = 0;
        for (auto x : id[i]) {
            for (auto y : es[x]) {
                if (find(find, x) == find(find, y)) continue;
                se.erase({a[y], y});
            }
            if (se.size()) {
                auto it = *se.begin();
                if (a[x] + it.first < mi) {
                    mi = a[x] + it.first;
                    p = it.second;
                }
            }
            for (auto y : es[x]) {
                if (find(find, x) == find(find, y)) continue;
                se.insert({a[y], y});
            }
        }
        if (p && find(find, i) != find(find, p)) {

```

```

        ++ cnt;
        ans += mi;
        f[find(find, i)] = find(find, p);
    }
    for (auto x : id[i]) {
        se.insert({a[x], x});
    }
}
return cnt;
};

while (boruvka());
for (int i = 1; i <= n; i++) {
    if (find(find, i) != find(find, 1)) {
        cout << -1 << "\n";
        return;
    }
}
cout << ans << "\n";
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(0);
    int _T = 1;
    cin >> _T;
    while (_T--) solve();
    return 0;
}

```

10. 树形换根 DP

适用场景

求每个点作为根时的答案，或者求“每个点到某类点的最远 / 最近距离”等信息。

这份模板解决的问题

有若干个被标记点，定义 f_i 为点 i 到所有被标记点距离的最大值，要求所有 f_i 的最小值。

核心思路

- `down[u]`：从 `u` 往子树内走，能到达标记点的最远距离
- `up[u]`：从 `u` 往父亲方向及兄弟子树方向走，能到达标记点的最远距离

最终每个点答案为：

$$\max(\text{down}[i], \text{up}[i])$$

复杂度

$$O(n)$$

```
#include<bits/stdc++.h>
```

```

using namespace std;
using ll = long long;

const int inf = 0x3f3f3f3f;

void solve() {
    int n, m;
    cin >> n >> m;
    vector<int> a(n + 10, 0);
    for (int i = 1; i <= m; i++) {
        int x;
        cin >> x;
        a[x] = 1;
    }

    vector<vector<int>> es(n + 10);
    for (int i = 1; i < n; i++) {
        int u, v;
        cin >> u >> v;
        es[u].push_back(v);
        es[v].push_back(u);
    }

    vector<int> down(n + 10, -inf), up(n + 10, -inf);

    auto dfs1 = [&](auto && self, int u, int fa) -> int {
        int t = -inf;
        if (a[u]) t = 0;
        for (auto it : es[u]) {
            if (it == fa) continue;
            t = max(t, self(self, it, u) + 1);
        }
        return down[u] = t;
    };

    auto dfs2 = [&](auto && self, int u, int fa) -> void {
        int d1 = -inf, d2 = -inf;
        if (a[u]) d1 = 0;
        for (auto it : es[u]) {
            if (it == fa) continue;
            int t = down[it] + 1;
            if (t > d1) {
                d2 = d1;
                d1 = t;
            } else if (t > d2) {
                d2 = t;
            }
        }

        for (auto it : es[u]) {
            if (it == fa) continue;
            int val = d1;

```

```

        if (down[it] + 1 == val) val = d2;
        up[it] = max({up[it], up[u] + 1, val + 1});
        self(self, it, u);
    }
};

dfs1(dfs1, 1, -1);
dfs2(dfs2, 1, -1);
int ans = inf;
for (int i = 1; i <= n; i++) {
    ans = min(ans, max(down[i], up[i]));
}
cout << ans << "\n";
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);
    int _T = 1;
    cin >> _T;
    while (_T--) solve();
    return 0;
}

```

11. 缩点

适用场景

有向图存在环，而题目要求“路径最优值”“最长路”“DP”等，通常先把强连通分量缩成 DAG 再做。

模板流程

1. Tarjan 求 SCC
2. 统计每个 SCC 的点权和
3. 建新图 DAG
4. 在 DAG 上拓扑 DP

复杂度

$O(n + m)$

注意点

- Tarjan 中回边更新写成 `low[u] = min(low[u], dfn[v])`
- 缩点后如果有重边，一般不影响 DAG DP，想去重可额外处理
- “点权最大路径和”这类题，初值通常设为 SCC 自身权值

```

#include<bits/stdc++.h>

using namespace std;

```

```

void solve() {
    int n, m;
    cin >> n >> m;

    // a 数组存储原图中每个点最开始的点权
    vector<int> a(n + 1, 0);
    for (int i = 1; i <= n; i++) {
        cin >> a[i];
    }

    // ===== 第 1 部分: Tarjan 算法相关的变量初始化 =====
    int timer = 0; // DFS 访问的时间戳计数器
    int cnt = 0; // 强连通分量 (SCC) 的总个数, 也是新图 (缩点后的 DAG) 的节点数

    vector<int> dfn(n + 1, 0); // dfn[i]: 节点 i 第一次被 DFS 访问到的时间戳
    vector<int> low(n + 1, 0); // low[i]: 节点 i 通过自己及子树, 能“绕回”到的最早的时间戳
    vector<int> vis(n + 1, 0); // vis[i]: 标记节点 i 是否正在当前的 DFS 栈中

    // ===== 第 2 部分: 缩点 (打包) 相关的变量初始化 =====
    vector<int> scc_id(n + 1, 0); // scc_id[i]: 原图中的节点 i, 被缩点后属于第几个超级节点
    vector<int> scc_w(n + 1, 0); // scc_w[i]: 第 i 个超级节点的总权值 (包含了缩在一起的所有原节点的权值)

    vector<vector<int>> es(n + 1); // 存储原图的有向边

    for (int i = 1; i <= m; i++) {
        int u, v;
        cin >> u >> v;
        es[u].push_back(v);
    }

    stack<int> sta; // Tarjan 算法所需的栈, 用来保存当前搜索路径上的点

    // ===== 第 3 部分: 核心 Tarjan 找环缩点函数 =====
    auto tarjan = [&](auto && self, int u) -> void {
        // 刚走到 u, 打上时间戳, 并初始化 low 为当前时间戳
        dfn[u] = low[u] = ++ timer;
        sta.push(u); // 入栈
        vis[u] = 1; // 标记 u 正在栈中

        // 遍历从 u 出发的所有有向边 u -> v
        for (auto v : es[u]) {
            if (!dfn[v]) {
                // 情况 A: v 还没被访问过, 是一条“树枝边”
                self(self, v); // 继续往下递归搜索 v
                // v 搜索完毕回溯时, 看看 v 能不能到达更早的点, 如果能, u 也能
                low[u] = min(low[u], low[v]);
            } else if (vis[v]) {
                // 情况 B: v 已经被访问过, 且 v 还在栈里!
                // 这说明我们顺着有向边走出了一个“环” (或者走到了祖先节点)。
                // 这时直接用 v 的初始时间戳来更新 u 的 low 值
                low[u] = min(low[u], dfn[v]);
            }
        }
    };
}

```

```

// (隐含的情况 C: v 访问过且不在栈中, 说明 v 属于之前已经找出的另一个 SCC, 也就是“横叉边”, 直接
忽略)
}

// 关键点: 当我们遍历完 u 的所有出边, 发现 low[u] 依然等于 dfn[u]
// 说明 u 及其子树无法绕回比 u 更早的节点了。
// 那么 u 就是当前这个强连通分量 (SCC) 在搜索树上的“最高点/根节点”。
if (dfn[u] == low[u]) {
    cnt++; // 发现了一个新的强连通分量 (新的超级节点诞生)
    vector<int> comp; // 这是一个多余变量, 代码里没用上, 可以删掉

    // 把栈里在 u 之后的点全部弹出来, 它们都属于这个新的超级节点 cnt
    while (1) {
        int x = sta.top();
        sta.pop();
        vis[x] = 0; // 出栈了, 取消标记

        scc_id[x] = cnt; // 登记身份: 原节点 x 属于第 cnt 个超级节点
        scc_w[cnt] += a[x]; // 累加财富: 把 x 的点权加到超级节点 cnt 上

        if (x == u) break; // 弹到 u 自己就可以停止了
    }
}

};

// 遍历原图所有点, 防范图不连通 (有多个分离的子图) 的情况
for (int i = 1; i <= n; i++) {
    if (!dfn[i]) tarjan(tarjan, i);
}

// ===== 第 4 部分: 重新建图 (构建 DAG) =====
vector<vector<int>> dag(cnt + 1); // dag: 缩点后的新图, 一共 cnt 个节点
vector<int> degree(cnt + 1, 0); // degree[i]: 记录新图中第 i 个超级节点的入度

// 遍历原图的每一条边, 尝试将其映射到新图上
for (int i = 1; i <= n; i++) {
    for (auto v : es[i]) {
        // 如果边 i -> v 的两端, 现在不再同一个超级节点里 (说明不是环内的边)
        if (scc_id[v] != scc_id[i]) {
            // 在新图连一条对应的边: 超级节点 scc_id[i] 指向 超级节点 scc_id[v]
            dag[scc_id[i]].push_back(scc_id[v]);
            degree[scc_id[v]]++; // 目标超级节点的入度 + 1
        }
        // 如果 scc_id[v] == scc_id[i], 说明这条边在环内部, 缩点后变成了毫无意义的自环, 直接丢弃
    }
}

// ===== 第 5 部分: 拓扑排序 + DAG 上的动态规划 =====
// dp[i]: 表示顺着有向边走到超级节点 i 时, 能累积的最大权值和
vector<int> dp(cnt + 1, 0);
queue<int> q;

// 拓扑排序第一步: 把所有入度为 0 的节点 (即所有可能的起点) 加入队列

```

```

for (int i = 1; i <= cnt; i++) {
    if (degree[i] == 0) {
        q.push(i);
        dp[i] = scc_w[i]; // 走到起点的初始最大权值，就是起点自身的权值
    }
}

// 开始 BFS 拓扑排序
while (q.size()) {
    int u = q.front();
    q.pop();

    // 遍历从当前节点 u 出发的所有新图边 u -> v
    for (auto v : dag[u]) {
        // 状态转移方程核心：
        // 到达 v 的最大收益 = max(到达 v 原本的最大收益，从 u 走过来累计的收益 + v 自身的收益)
        dp[v] = max(dp[v], dp[u] + scc_w[v]);

        // 把这条边“删掉”，即目标节点的入度减 1
        // 如果入度减为 0，说明到达 v 的所有前置路线都已经算完了，v 可以入队了
        if (-- degree[v] == 0) {
            q.push(v);
        }
    }
}

// ===== 第 6 部分：统计最终答案 =====
int ans = 0;
// 路径可以在任何一个超级节点结束，所以我们需要在所有的 dp[i] 中挑一个最大的
for (int i = 1; i <= cnt; i++) {
    ans = max(ans, dp[i]);
}

cout << ans << "\n";
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);
    int _T = 1;
    // cin >> _T;
    while (_T--) solve();
    return 0;
}

```

四、数学知识

1. 数论

1.1 筛质数

埃氏筛

作用：求 $1 \sim n$ 中质数个数，或预处理哪些数是质数。

时间复杂度： $O(n \log \log n)$

```
const int N = 1e6 + 10;
int p[N], cnt;
bool st[N];

void get_primes(int n) {
    st[0] = st[1] = true;
    cnt = 0;
    for (int i = 2; i <= n; i++) {
        if (st[i]) continue;
        p[cnt++] = i;
        for (int j = 2; j <= n / i; j++) {
            st[i * j] = true;
        }
    }
}
```

线性筛

作用：更常用，能在线性时间内筛出所有质数。

时间复杂度： $O(n)$

```
const int N = 1e6 + 10;
int p[N], cnt;
bool st[N];

void get_primes(int n) {
    st[0] = st[1] = true;
    cnt = 0;
    for (int i = 2; i <= n; i++) {
        if (!st[i]) p[cnt++] = i;
        for (int j = 0; j < cnt && p[j] <= n / i; j++) {
            st[i * p[j]] = true;
            if (i % p[j] == 0) break;
        }
    }
}
```

1.2 约数个数

若

$$n = p_1^{a_1} p_2^{a_2} \cdots p_k^{a_k}$$

那么它的约数个数为：

$$(a_1 + 1)(a_2 + 1) \cdots (a_k + 1)$$

如果题目给你多个数，要求这些数的**乘积的约数个数**，就把所有质因子的指数累加起来再算。

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

const int mod = 1e9 + 7;
map<int, int> cnt;

void divide(int x) {
    for (int i = 2; i <= x / i; i++) {
        while (x % i == 0) {
            x /= i;
            cnt[i]++;
        }
    }
    if (x > 1) cnt[x]++;
}

int main() {
    int n;
    cin >> n;
    while (n--) {
        int x;
        cin >> x;
        divide(x);
    }

    ll res = 1;
    for (auto [p, a] : cnt) {
        res = res * (a + 1) % mod;
    }
    cout << res << "\n";
    return 0;
}
```

1.3 约数之和

若

$$n = p_1^{a_1} p_2^{a_2} \cdots p_k^{a_k}$$

那么约数之和为：

$$(1 + p_1 + p_1^2 + \cdots + p_1^{a_1})(1 + p_2 + p_2^2 + \cdots + p_2^{a_2}) \cdots (1 + p_k + p_k^2 + \cdots + p_k^{a_k})$$

如果是多个数的乘积，同样先统计总质因子指数。

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

const int mod = 1e9 + 7;
map<int, int> cnt;

void divide(int x) {
    for (int i = 2; i <= x / i; i++) {
        while (x % i == 0) {
            x /= i;
            cnt[i]++;
        }
    }
    if (x > 1) cnt[x]++;
}

int main() {
    int n;
    cin >> n;
    while (n--) {
        int x;
        cin >> x;
        divide(x);
    }

    ll res = 1;
    for (auto [p, a] : cnt) {
        ll t = 1;
        while (a--) t = (t * p + 1) % mod;
        res = res * t % mod;
    }

    cout << res << "\n";
    return 0;
}
```

1.4 阶乘分解

要求 $n!$ 中质因子 p 的次数, 用:

$$\left\lfloor \frac{n}{p} \right\rfloor + \left\lfloor \frac{n}{p^2} \right\rfloor + \left\lfloor \frac{n}{p^3} \right\rfloor + \dots$$

```
#include <bits/stdc++.h>
using namespace std;

const int N = 1e6 + 10;
int p[N], cnt;
bool st[N];

void get_primes(int n) {
    cnt = 0;
    st[0] = st[1] = true;
    for (int i = 2; i <= n; i++) {
        if (!st[i]) p[cnt++] = i;
        for (int j = 0; j < cnt && p[j] <= n / i; j++) {
            st[i * p[j]] = true;
            if (i % p[j] == 0) break;
        }
    }
}

int get(int n, int p) {
    int res = 0;
    while (n) {
        res += n / p;
        n /= p;
    }
    return res;
}

int main() {
    int n;
    cin >> n;
    get_primes(n);

    for (int i = 0; i < cnt; i++) {
        cout << p[i] << " " << get(n, p[i]) << "\n";
    }
    return 0;
}
```

1.5 gcd / exgcd

最大公约数

用途：求 $\gcd(a, b)$ 。

```
int gcd(int a, int b) {
    while (b != 0) {
        int t = a % b;
        a = b;
        b = t;
    }
    return a;
}
```

扩展欧几里得

用途：求解

$$ax + by = s$$

也可用于求逆元、线性同余、丢番图方程。

结论：

- 方程有解当且仅当 $s \bmod \gcd(a, b) = 0$
- 若 (x_0, y_0) 是一组特解，则通解为

$$x = x_0 + k \frac{b}{g}, \quad y = y_0 - k \frac{a}{g}, \quad g = \gcd(a, b)$$

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using i128 = __int128;

i128 div_floor(i128 a, i128 b) {
    i128 res = a / b;
    if (a % b != 0 && ((a < 0) ^ (b < 0))) res--;
    return res;
}

i128 div_ceil(i128 a, i128 b) {
    return -div_floor(-a, b);
}

ll exgcd(ll a, ll b, i128& x, i128& y) {
    if (b == 0) {
        x = 1, y = 0;
        return a;
    }
    ll d = exgcd(b, a % b, y, x);
    y -= a / b * x;
```

```

    return d;
}

void solve() {
    ll s1, a1, b1;
    cin >> s1 >> a1 >> b1;
    i128 x = 0, y = 0;
    ll g = exgcd(a1, b1, x, y);
    if (s1 % g != 0) {
        cout << "No\n";
        return;
    }

    ll a = a1 / g;
    ll b = b1 / g;
    ll s = s1 / g;
    x *= s;
    y *= s;

    i128 k1 = div_ceil(-x, b);
    i128 k2 = div_floor(y, a);
    if (k1 > k2) {
        cout << "No\n";
        return;
    }

    i128 l = k1, r = k2;
    while (l + 1 < r) {
        i128 mid = (l + r) >> 1;
        i128 x1 = x + mid * b;
        i128 y1 = y - mid * a;
        if (x1 >= y1) r = mid;
        else l = mid;
    }

    bool ok = false;
    i128 best = -1;
    ll ansx = 0, ansy = 0;
    for (i128 i = l - 1; i <= r; i++) {
        if (i < k1 || i > k2) continue;
        i128 nx = x + i * b;
        i128 ny = y - i * a;
        if (!ok || max(nx, ny) < best) {
            best = max(nx, ny);
            ansx = nx;
            ansy = ny;
            ok = true;
        }
    }
    cout << "Yes\n";
    cout << ansx << " " << ansy << "\n";
}

```

扩展欧拉定理

用途：求 $a^b \bmod m$ ，其中 b 极大，以字符串给出。

核心：

- 先求 $\varphi(m)$
- 若指数足够大，则降幂为 $b \bmod \varphi(m) + \varphi(m)$

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;

string s;
ll a, m;

ll get_phi(ll m) {
    ll res = m;
    for (ll i = 2; i * i <= m; i++) {
        if (m % i == 0) {
            res = res / i * (i - 1);
            while (m % i == 0) m /= i;
        }
    }
    if (m > 1) res = res / m * (m - 1);
    return res;
}

ll depow(ll phi) {
    ll b = 0;
    bool flag = 0;
    for (char ch : s) {
        b = b * 10 + (ch - '0');
        if (b >= phi) flag = 1, b %= phi;
    }
    if (flag) b += phi;
    return b;
}

ll qmi(ll a, ll b) {
    ll res = 1;
    while (b) {
        if (b & 1) res = res * a % m;
        a = a * a % m;
        b >>= 1;
    }
    return res;
}

void solve() {
    cin >> a >> m >> s;
    ll ans = depow(get_phi(m));
    cout << qmi(a, ans) << "\n";
}
```

```
}
```

1.6 数论分块

向下取整分块

用途：处理

$$\sum_{i=1}^n \left\lfloor \frac{n}{i} \right\rfloor$$

或所有形如 $\left\lfloor \frac{n}{i} \right\rfloor$ 值域分段不变的问题。

复杂度： $O(\sqrt{n})$

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;

void solve() {
    ll n;
    cin >> n;
    ll res = 0;
    for (ll l = 1, r; l <= n; l = r + 1) {
        r = n / (n / l);
        res += 1LL * (r - l + 1) * (n / l);
    }
    cout << res << "\n";
}
```

向上取整分块

结论：

$$\left\lceil \frac{n}{i} \right\rceil = \left\lfloor \frac{n-1}{i} \right\rfloor + 1$$

注意：当分块转化时要注意 $k = 1$ 的特殊情况，避免除以 0。

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;

void solve() {
    ll n;
    cin >> n;
    ll res = 0;
    for (ll l = 1, r; l <= n; l = r + 1) {
        ll k = (n + l - 1) / l;
        if (k == 1) r = n;
        else {
            r = (n - 1) / (k - 1);
        }
    }
}
```

```

        r = min(r, n);
    }
    res += (r - 1 + 1) * k;
}
cout << res << "\n";
}

```

1.7 Legendre 定理

用途： 求质数 p 在 $n!$ 中的次数。

公式：

$$v_p(n!) = \sum_{k \geq 1} \left\lfloor \frac{n}{p^k} \right\rfloor$$

常用于组合数分解质因数。

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;

const int N = 5000;
vector<int> primes;

void init() {
    vector<bool> ip(N + 1, true);
    for (int i = 2; i <= N; ++i) {
        if (ip[i]) primes.push_back(i);
        for (int p : primes) {
            if (i * p > N) break;
            ip[i * p] = false;
            if (i % p == 0) break;
        }
    }
}

ll qmi(ll a, ll b, ll mod) {
    ll ans = 1;
    while (b) {
        if (b & 1) ans = ans * a % mod;
        a = a * a % mod;
        b >>= 1;
    }
    return ans;
}

int legendre(int n, int p) {
    int cnt = 0;
    while (n) {
        n /= p;
        cnt += n;
    }
}

```

```

    return cnt;
}

void solve() {
    int t, mod;
    cin >> t >> mod;
    while (t--) {
        int n;
        cin >> n;
        vector<int> a(n + 1);
        ll s = 0;
        for (int i = 1; i <= n; i++) {
            cin >> a[i];
            s += a[i];
        }
        ll ans = 1;
        for (int p : primes) {
            if (p > s) break;
            int alpha = legendre(s, p);
            for (int i = 1; i <= n; i++) alpha -= legendre(a[i], p);
            ans = ans * qmi(p, alpha, mod) % mod;
        }
        cout << ans << "\n";
    }
}

```

1.8 威尔逊定理

若 p 为素数, 则

$$(p-1)! \equiv -1 \pmod{p}$$

即 $p \mid (p-1)! + 1$ 。

1.9 筛质数 + 求 mu

```

const int N = 1e6 + 10;
int mu[N], primes[N], vis[N], tot;

void get_mu(int n) {
    mu[1] = 1;
    for (int i = 2; i <= n; i++) {
        if (!vis[i]) {
            primes[++tot] = i;
            mu[i] = -1;
        }
        for (int j = 1; j <= tot && primes[j] * i <= n; j++) {
            int p = primes[j];
            vis[i * p] = 1;
            if (i % p == 0) {
                mu[i * p] = 0;
                break;
            } else {

```

```

        mu[i * p] = -mu[i];
    }
}
}
}

```

1.10 矩阵快速幂

普通矩阵幂

用途： 线性递推、路径计数、状态转移。

复杂度： $O(n^3 \log k)$

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;

const int N = 110;
const int mod = 1e9 + 7;

struct Matrix {
    ll c[N][N];
    Matrix() { memset(c, 0, sizeof c); }
};

Matrix A, res;
ll n, k;

Matrix operator * (Matrix &x, Matrix &y) {
    Matrix t;
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= n; j++)
            for (int k = 1; k <= n; k++)
                t.c[i][j] = (t.c[i][j] + x.c[i][k] * y.c[k][j]) % mod;
    return t;
}

void quickpow(ll k) {
    for (int i = 1; i <= n; i++) res.c[i][i] = 1;
    while (k) {
        if (k & 1) res = res * A;
        A = A * A;
        k >>= 1;
    }
}

```

min-plus 矩阵快速幂

用途：恰好走 k 步最短路。

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;

const int N = 110;
const ll INF = 0x3f3f3f3f3f3f3f;

struct Matrix {
    ll c[N][N];
    Matrix() {
        for (int i = 1; i <= 100; i++)
            for (int j = 1; j <= 100; j++)
                c[i][j] = INF;
    }
};

Matrix A, res;
ll n, k;

Matrix operator * (Matrix &x, Matrix &y) {
    Matrix t;
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= n; j++)
            for (int k = 1; k <= n; k++)
                t.c[i][j] = min(t.c[i][j], x.c[i][k] + y.c[k][j]);
    return t;
}

void quickpow(ll k) {
    for (int i = 1; i <= n; i++) res.c[i][i] = 0;
    while (k) {
        if (k & 1) res = res * A;
        A = A * A;
        k >>= 1;
    }
}
```

1.11 支持除 0 的模数

用途：维护一个乘积，支持乘 / 除，并允许因子为 0。

```
ll qmi(ll a, ll b) {
    ll res = 1;
    while (b) {
        if (b & 1) res = res * a % mod;
        a = a * a % mod;
        b >>= 1;
    }
}
```

```

    return res;
}

ll inv(ll x) {
    return qmi(x, mod - 2);
}

struct Z {
    int zero = 0;
    ll res = 1;

    void mul(ll y) {
        y %= mod;
        if (y < 0) y += mod;
        if (y != 0) res = res * y % mod;
        else zero++;
    }

    void div(ll y) {
        y %= mod;
        if (y < 0) y += mod;
        if (y != 0) res = res * inv(y) % mod;
        else zero--;
    }

    ll val() {
        if (zero > 0) return 0;
        return res;
    }
};

```

2. 线性代数

2.1 高斯消元（实数域）

适用于形如：

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n = b_1 \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n = b_2 \\ \dots \end{cases}$$

区分三种情况：唯一解 / 无穷多解 / 无解。

```

#include <bits/stdc++.h>
using namespace std;

const int N = 110;
const double eps = 1e-8;
double a[N][N];
int n;

// 0: 唯一解

```

```

// 1: 无穷多解
// 2: 无解
int gauss() {
    int r = 1;
    for (int c = 1; c <= n; c++) {
        int t = r;
        for (int i = r; i <= n; i++) {
            if (fabs(a[i][c]) > fabs(a[t][c])) t = i;
        }

        if (fabs(a[t][c]) < eps) continue;

        for (int j = c; j <= n + 1; j++) swap(a[t][j], a[r][j]);

        for (int j = n + 1; j >= c; j--) a[r][j] /= a[r][c];

        for (int i = r + 1; i <= n; i++) {
            if (fabs(a[i][c]) > eps) {
                for (int j = n + 1; j >= c; j--) {
                    a[i][j] -= a[r][j] * a[i][c];
                }
            }
        }
        r++;
    }

    if (r <= n) {
        for (int i = r; i <= n; i++) {
            if (fabs(a[i][n + 1]) > eps) return 2;
        }
        return 1;
    }

    for (int i = n - 1; i >= 1; i--) {
        for (int j = n; j >= i + 1; j--) {
            a[i][n + 1] -= a[i][j] * a[j][n + 1];
        }
    }

    for (int i = 1; i <= n; i++) {
        if (fabs(a[i][n + 1]) < eps) a[i][n + 1] = 0;
    }
    return 0;
}

int main() {
    cin >> n;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n + 1; j++) {
            cin >> a[i][j];
        }
    }
}

```

```

int flag = gauss();
if (flag == 0) {
    for (int i = 1; i <= n; i++) printf("%.2f\n", a[i][n + 1]);
} else if (flag == 1) {
    cout << "Infinite group solutions\n";
} else {
    cout << "No solution\n";
}
return 0;
}

```

2.2 高斯消元（异或方程组）

这种题系数只有 0/1，运算改成异或。

```

#include <bits/stdc++.h>
using namespace std;

const int N = 110;
int a[N][N];
int n;

// 0: 唯一解
// 1: 多组解
// 2: 无解
int gauss() {
    int r = 1;
    for (int c = 1; c <= n; c++) {
        int t = r;
        while (t <= n && a[t][c] == 0) t++;
        if (t > n) continue;

        for (int j = c; j <= n + 1; j++) swap(a[t][j], a[r][j]);

        for (int i = r + 1; i <= n; i++) {
            if (a[i][c]) {
                for (int j = c; j <= n + 1; j++) {
                    a[i][j] ^= a[r][j];
                }
            }
        }
        r++;
    }

    if (r <= n) {
        for (int i = r; i <= n; i++) {
            if (a[i][n + 1]) return 2;
        }
        return 1;
    }
}

```

```

    for (int i = n - 1; i >= 1; i--) {
        for (int j = n; j >= i + 1; j--) {
            a[i][n + 1] ^= (a[i][j] & a[j][n + 1]);
        }
    }
    return 0;
}

int main() {
    cin >> n;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n + 1; j++) {
            cin >> a[i][j];
        }
    }

    int flag = gauss();
    if (flag == 0) {
        for (int i = 1; i <= n; i++) cout << a[i][n + 1] << "\n";
    } else if (flag == 1) {
        cout << "Multiple sets of solutions\n";
    } else {
        cout << "No solution\n";
    }
    return 0;
}

```

2.3 线性基

标准模板

```

struct LB {
    using ll = long long;
    static const int M = 62;
    ll p[M + 1];
    ll d[M + 1];
    int cnt;
    bool flag;
    bool rebuilt;

    LB() {
        memset(p, 0, sizeof(p));
        memset(d, 0, sizeof(d));
        cnt = 0;
        flag = false;
        rebuilt = false;
    }

    bool insert(ll x) {
        for (int i = M; i >= 0; i--) {
            if (x & (1LL << i)) {
                if (!p[i]) {

```

```

        p[i] = x;
        cnt++;
        return true;
    }
    x ^= p[i];
}
}
flag = true;
return false;
}

ll qmax(ll res = 0) {
    for (int i = M; i >= 0; i--) {
        if ((res ^ p[i]) > res) res ^= p[i];
    }
    return res;
}

ll qmin() {
    if (flag) return 0;
    for (int i = 0; i <= M; i++) {
        if (p[i]) return p[i];
    }
    return 0;
}

bool check(ll x) {
    for (int i = M; i >= 0; i--) {
        if (x & (1LL << i)) {
            if (!p[i]) return false;
            x ^= p[i];
        }
    }
    return x == 0;
}

void rebuild() {
    for (int i = M; i >= 0; i--) {
        for (int j = i - 1; j >= 0; j--) {
            if (p[i] & (1LL << j)) p[i] ^= p[j];
        }
    }
    cnt = 0;
    for (int i = 0; i <= M; i++) {
        if (p[i]) d[cnt++] = p[i];
    }
    rebuilt = true;
}

ll qk(ll k) {
    if (!rebuilt) rebuild();
    if (flag) k--;
    if (k >= (1LL << cnt)) return -1;

```

```

        ll res = 0;
        for (int i = 0; i < cnt; i++) {
            if (k & (1LL << i)) res ^= d[i];
        }
        return res;
    }
};

```

带路径还原

```

struct PathLB {
    using ll = long long;
    static const int M = 62;
    ll p[M + 1];
    int id[M + 1];
    ll mask[M + 1];

    PathLB() {
        memset(p, 0, sizeof(p));
        memset(id, 0, sizeof(id));
        memset(mask, 0, sizeof(mask));
    }

    void insert(ll x, int pos) {
        ll m = (1LL << pos);
        for (int i = M; i >= 0; i--) {
            if (x & (1LL << i)) {
                if (!p[i]) {
                    p[i] = x;
                    id[i] = pos;
                    mask[i] = m;
                    return;
                }
                x ^= p[i];
                m ^= mask[i];
            }
        }
    }

    vector<int> get_path(ll res) {
        vector<int> path;
        ll tar = 0;
        for (int i = M; i >= 0; i--) {
            if (res & (1LL << i)) {
                if (!p[i]) return {};
                res ^= p[i];
                tar ^= mask[i];
            }
        }
        if (res != 0) return {};
        for (int i = 0; i <= 60; i++) {
            if (tar & (1LL << i)) path.push_back(i);
        }
    }
};

```

```

    }
    return path;
}
};

```

线性基合并

```

void merge(LB &A, const LB &B) {
    for (int i = 0; i <= LB::M; i++) {
        if (B.p[i]) A.insert(B.p[i]);
    }
}

```

3. 博弈论

3.1 SG 函数

```

// sg函数的异或和不为0时，先手必胜
// sg(x) = mex(sg(x的直接后继))
// x没有后继时，认为sg(x) = 0
// mex(set) = set外最小的非负整数

```

解释

对于一个状态 x ：

- 先列出它一步能走到的所有状态，叫“直接后继”
- 把这些后继状态的 sg 值收集起来
- $sg(x)$ 就是这些值的 mex

若游戏由若干个**相互独立**的子游戏组成，则总状态的 SG 值为各子游戏 SG 值的异或和：

$$sg = sg_1 \oplus sg_2 \oplus \cdots \oplus sg_n$$

结论：

- 异或和 $\neq 0$ ，先手必胜
- 异或和 $= 0$ ，先手必败

3.2 集合 Nim

适用于：每次只能拿 $s[i]$ 个石子。

```

#include <bits/stdc++.h>
using namespace std;

const int N = 110;
const int M = 10010;

```

```

int s[N], f[M];
int k;

int sg(int x) {
    if (f[x] != -1) return f[x];

    set<int> st;
    for (int i = 0; i < k; i++) {
        if (x >= s[i]) st.insert(sg(x - s[i]));
    }

    for (int i = 0; ; i++) {
        if (!st.count(i)) return f[x] = i;
    }
}

int main() {
    cin >> k;
    for (int i = 0; i < k; i++) cin >> s[i];

    int n;
    cin >> n;

    memset(f, -1, sizeof f);

    int res = 0;
    while (n--) {
        int x;
        cin >> x;
        res ^= sg(x);
    }

    cout << (res ? "Yes" : "No") << "\n";
    return 0;
}

```

3.3 拆分 Nim

若一个状态可以拆成两个独立子状态 i, j ，则：

$$sg(x \rightarrow (i, j)) = sg(i) \oplus sg(j)$$

```

#include <bits/stdc++.h>
using namespace std;

const int N = 110;
int f[N];

int sg(int x) {
    if (f[x] != -1) return f[x];

    set<int> st;

```

```

for (int i = 0; i < x; i++) {
    for (int j = 0; j < x; j++) {
        st.insert(sg(i) ^ sg(j));
    }
}

for (int i = 0; ; i++) {
    if (!st.count(i)) return f[x] = i;
}
}

int main() {
    int n;
    cin >> n;

    memset(f, -1, sizeof f);

    int res = 0;
    while (n--) {
        int x;
        cin >> x;
        res ^= sg(x);
    }

    cout << (res ? "Yes" : "No") << "\n";
    return 0;
}

```

4. 组合数学

4.1 组合数

杨辉三角

递推:

$$C_n^m = C_{n-1}^m + C_{n-1}^{m-1}$$

$O(n^2)$ 求组合数

```
void solve() {
    int n, m;
    cin >> n >> m;
    vector<vector<long long>> f(n + 10, vector<long long>(n + 10, 0));
    for (int i = 0; i <= n; i++) {
        for (int j = 0; j <= i; j++) {
            if (i == 0 || j == 0) f[i][j] = 1;
            else f[i][j] = f[i - 1][j - 1] + f[i - 1][j];
        }
    }
    cout << f[n][m];
}
```

阶乘逆元预处理

用途：模质数下大量求组合数。

复杂度：预处理 $O(N)$ ，单次查询 $O(1)$ 。

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;

const int N = 1e6 + 10;
const ll MOD = 998244353LL;

ll fact[N], invfact[N];

ll qmi(ll a, ll b, ll mod) {
    ll res = 1;
    a = (a % mod + mod) % mod;
    while (b) {
        if (b & 1) res = res * a % mod;
        a = a * a % mod;
        b >>= 1;
    }
    return res;
}

void init() {
    int n = 1e6;
    fact[0] = 1;
    for (int i = 1; i <= n; i++) fact[i] = fact[i - 1] * i % MOD;
    invfact[n] = qmi(fact[n], MOD - 2, MOD);
    for (int i = n; i >= 1; i--) invfact[i - 1] = invfact[i] * i % MOD;
}

ll C(ll n, ll m) {
    if (n < m || m < 0) return 0LL;
    return fact[n] * invfact[m] % MOD * invfact[n - m] % MOD;
}
```

```

}

vector<ll> build_prefix_comb(int n, int limit) {
    vector<ll> g(n + 1, 0);
    if (limit >= 0) {
        g[0] = 1;
        for (int c = 1; c <= n; c++) {
            ll term = C(c - 1, limit);
            g[c] = (2 * g[c - 1] % MOD - term + MOD) % MOD;
        }
    }
    return g;
}

```

4.2 Lucas 定理

适用场景：模数 p 是质数，且 n, m 很大。

公式：

$$\binom{n}{k} \equiv \prod_i \binom{n_i}{k_i} \pmod{p}$$

其中 n_i, k_i 为 p 进制位。

```

#include <bits/stdc++.h>
using namespace std;
using ll = long long;

ll qmi(ll a, ll n, ll mod) {
    ll res = 1;
    a = (a % mod + mod) % mod;
    while (n) {
        if (n & 1) res = a * res % mod;
        a = a * a % mod;
        n >>= 1;
    }
    return res;
}

ll C(ll n, ll m, ll mod) {
    if (m > n || n < 0 || m < 0) return 0LL;
    ll res = 1;
    for (int i = 1, j = n; i <= m; i++, j--) {
        res = res * j % mod;
        res = res * qmi(i, mod - 2, mod) % mod;
    }
    return res;
}

ll lucas(ll a, ll b, ll p) {
    if (a < p && b < p) return C(a, b, p);
    return C(a % p, b % p, p) * lucas(a / p, b / p, p) % p;
}

```

```

}

void solve() {
    ll n, m, mod;
    cin >> n >> m >> mod;
    cout << lucas(n + m, n, mod) << "\n";
}

```

4.3 插板法

核心模型： 将 n 个相同物品分给 k 个不同盒子。

- 每盒至少一个：

$$\binom{n-1}{k-1}$$

- 允许为空：

$$\binom{n+k-1}{k-1}$$

- 每盒至少 m 个：

$$\binom{n-km+k-1}{k-1}$$

- 每盒至少 m_i 个，设 $S = \sum m_i$ ：

$$\binom{n-S+k-1}{k-1}$$

4.4 多重集合排列

若总数为 n ，第 i 种元素重复 n_i 次，则排列数为

$$\frac{n!}{n_1!n_2!\cdots n_k!}$$

4.5 第二类斯特林数

含义： 将 n 个不同球放进 m 个不同盒子且每盒非空的方案数。

递推：

$$f[i][j] = j \cdot (f[i-1][j] + f[i-1][j-1])$$

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;
const ll MOD = 998244353;

void solve() {
    int n, m;
    cin >> n >> m;
}

```

```

vector<vector<ll>> f(n + 10, vector<ll>(m + 10, 0));
f[0][0] = 1;
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= m; j++) {
        f[i][j] = j * (f[i - 1][j] + f[i - 1][j - 1]) % MOD;
    }
}
cout << f[n][m];
}

```

4.6 分拆数

定义： $p(n, k)$ 表示把 n 拆成恰好 k 部分的方案数。

递推：

$$p(n, k) = p(n - 1, k - 1) + p(n - k, k)$$

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;
const int mod = 1e9 + 7;

void solve() {
    ll n, m;
    cin >> n >> m;
    vector<vector<ll>> dp(n + 10, vector<ll>(500 + 10, 0));
    dp[0][0] = 1;
    for (int i = 0; i <= n; i++) {
        for (int j = 1; j <= i && j <= 500; j++) {
            dp[i][j] = (dp[i - j][j] + dp[i - j][j - 1]) % mod;
        }
    }
    cout << dp[n][m] << "\n";
}

```

4.7 范德蒙德卷积

$$\sum_{i=0}^k \binom{n}{i} \binom{m}{k-i} = \binom{n+m}{k}$$

常见推论：

$$\sum_{i=0}^n \binom{n}{i}^2 = \binom{2n}{n}$$

$$\sum_{i=1}^n \binom{n}{i} \binom{n}{i-1} = \binom{2n}{n-1}$$

$$\sum_{i=0}^m \binom{n}{i} \binom{m}{i} = \binom{n+m}{m}$$

4.8 错位排列

前置知识补充：在 n 个元素的错排中，强制某个人传给特定另一个人的方案数，可以分类讨论为两人互传（剩下 $n - 2$ 错排）和两人不互传（相当于把这俩人绑成一个点，剩下 $n - 1$ 错排），总数为 $D_{n-1} + D_{n-2}$ 。

递推：

$$D_1 = 0, D_2 = 1, D_n = (n - 1)(D_{n-1} + D_{n-2})$$

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

ll get(ll x) {
    if (x == 1) return 0;
    if (x == 2) return 1;
    return (x - 1) * (get(x - 1) + get(x - 2));
}

void solve() {
    int n;
    cin >> n;
    cout << get(n);
}
```

4.9 卡特兰数

常见形式：

$$H_n = \frac{1}{n+1} \binom{2n}{n}$$
$$H_n = \binom{2n}{n} - \binom{2n}{n-1}$$
$$H_n = \frac{4n-2}{n+1} H_{n-1}$$

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

void solve() {
    int n;
    cin >> n;
    vector<ll> a(n + 10, 0);
    a[0] = 1;
    for (int i = 1; i <= n; i++) {
        a[i] = a[i - 1] * (4 * i - 2) / (i + 1);
    }
    cout << a[n] << "\n";
}
```

括号序列计数常用写法

```
11 ways(int rem, int h) {
    if (h < 0 || rem < 0) return 0;
    if (rem < h) return 0;
    if ((rem - h) & 1) return 0;

    int k = (rem - h) / 2;
    return (C(rem, k) - C(rem, k - 1) + MOD) % MOD;
}
```

5. 其他常用内容

5.1 分数结构体

```
struct FRAC {
    int a, b;
    FRAC(const int _a = 0, const int _b = 1) : a(_a), b(_b) {
        int g = __gcd(a, b);
        a /= g;
        b /= g;
        if (b < 0) {
            a = -a;
            b = -b;
        }
    }
    bool operator < (const FRAC& B) const {
        return (long long)a * B.b < (long long)B.a * b;
    }
    bool operator == (const FRAC& B) const {
        return (long long)a * B.b == (long long)B.a * b;
    }
};
```

5.2 C++ 常用数学函数

函数	作用
abs / fabs	绝对值
pow	幂
sqrt / cbrt	平方根 / 立方根
hypot	斜边长度
ceil / floor / round / trunc	各类取整
fmod / remainder	浮点取模

函数	作用
<code>exp / exp2</code>	指数
<code>log / log10 / log2</code>	对数
<code>sin / cos / tan</code>	三角函数
<code>asin / acos / atan / atan2</code>	反三角函数
<code>sinh / cosh / tanh</code>	双曲函数
<code>tgamma / lgamma</code>	Gamma 相关
<code>fmax / fmin</code>	最大 / 最小
<code>copysign</code>	复制符号
<code>fdim</code>	正差
<code>fma</code>	融合乘加

5.3 生成函数

普通生成函数

$$F(x) = \sum_{n=0}^{\infty} a_n x^n$$

指数生成函数

$$F(x) = \sum_{n=0}^{\infty} a_n \frac{x^n}{n!}$$

常见展开

$$\frac{1}{1-x} = 1 + x + x^2 + x^3 + \dots$$

$$\frac{1}{(1-x)^2} = 1 + 2x + 3x^2 + 4x^3 + \dots$$

$$e^x = \sum_{n=0}^{\infty} \frac{x^n}{n!}$$

$$e^{-x} = \sum_{n=0}^{\infty} \frac{(-1)^n x^n}{n!}$$

五、计算几何

1. 基础模板

1.1 浮点数安全比较

计算几何里通常不能直接写 `a == b`、`a < b`。因为浮点数存在误差，所以一般先写一个符号函数：

```
int sgn(double x) {
    const double eps = 1e-9;
    if (fabs(x) < eps) return 0;
    return x < 0 ? -1 : 1;
}
```

常见对应关系：

普通写法	安全写法	含义
<code>a == b</code>	<code>sgn(a - b) == 0</code>	认为二者相等
<code>a != b</code>	<code>sgn(a - b) != 0</code>	认为二者不等
<code>a < b</code>	<code>sgn(a - b) < 0</code>	<code>a</code> 严格小于 <code>b</code>
<code>a <= b</code>	<code>sgn(a - b) <= 0</code>	<code>a</code> 小于或近似等于 <code>b</code>
<code>a > b</code>	<code>sgn(a - b) > 0</code>	<code>a</code> 严格大于 <code>b</code>
<code>a >= b</code>	<code>sgn(a - b) >= 0</code>	<code>a</code> 大于或近似等于 <code>b</code>

1.2 点、向量、叉积、点积

这是最常用的一套基础结构。

```
#include<bits/stdc++.h>
using namespace std;

int sgn(double x) {
    const double eps = 1e-9;
    if (fabs(x) < eps) return 0;
    return x < 0 ? -1 : 1;
}

struct Point {
    double x, y;
    Point(double X = 0, double Y = 0): x(X), y(Y) {}

    Point operator + (const Point &B) const {
        return Point(x + B.x, y + B.y);
    }
}
```

```

Point operator - (const Point &B) const {
    return Point(x - B.x, y - B.y);
}
Point operator * (double k) const {
    return Point(x * k, y * k);
}
Point operator / (double k) const {
    return Point(x / k, y / k);
}
};

double Cross(Point A, Point B) {
    return A.x * B.y - A.y * B.x;
}

double Dot(Point A, Point B) {
    return A.x * B.x + A.y * B.y;
}

double dis(Point a, Point b) {
    return sqrt((a.x - b.x) * (a.x - b.x) + (a.y - b.y) * (a.y - b.y));
}

```

1.3 三点叉积与方向判断

对于三点 `a`, `b`, `c`, 常写成:

```

double cross(Point a, Point b, Point c) {
    return (b.x - a.x) * (c.y - a.y) - (b.y - a.y) * (c.x - a.x);
}

```

几何意义:

- `cross(a, b, c) > 0`: `c` 在有向直线 `ab` 左侧
- `cross(a, b, c) < 0`: `c` 在有向直线 `ab` 右侧
- `cross(a, b, c) = 0`: 三点共线

1.4 求两直线交点

设两条直线分别经过点 `a`, `b` 和 `c`, `d`

那么交点可以写成:

```

// 是否平行
bool parallel(Point a, Point b, Point c, Point d) {
    return sgn(Cross(b - a, d - c)) == 0;
}

// 两直线交点

```

```

Point getNode(Point a, Point b, Point c, Point d) {
    Point u = b - a;
    Point v = d - c;
    double t = Cross(c - a, v) / Cross(u, v);
    return a + u * t;
}

if (!parallel(a, b, c, d)) {
    Point p = getNode(a, b, c, d);
}

```

2. 基础判断

2.1 点在线段上

这是非常常见的判断。

```

#include<bits/stdc++.h>
using namespace std;

int sgn(double x) {
    if (fabs(x) < 1e-9) return 0;
    return x < 0 ? -1 : 1;
}

struct Point {
    double x, y;
    Point(double X = 0, double Y = 0): x(X), y(Y) {}
    Point operator + (Point B) { return Point(x + B.x, y + B.y); }
    Point operator - (Point B) { return Point(x - B.x, y - B.y); }
};

double Cross(Point A, Point B) {
    return A.x * B.y - A.y * B.x;
}

double Dot(Point A, Point B) {
    return A.x * B.x + A.y * B.y;
}

struct Line {
    Point p1, p2;
    Line(Point P1, Point P2): p1(P1), p2(P2) {}
};

bool Point_on_seg(Point p, Line v) {
    return sgn(Cross(p - v.p1, v.p2 - v.p1)) == 0
        && sgn(Dot(p - v.p1, p - v.p2)) <= 0;
}

```

判断逻辑：

- 先用叉积判断是否共线
- 再用点积判断点是否夹在线段两端之间

2.2 点在多边形内

这个板子可以判断：

- 点在顶点上
- 点在边上
- 点在内部
- 点在外部

```
#include<bits/stdc++.h>
using namespace std;

int sgn(double x) {
    if (fabs(x) < 1e-9) return 0;
    return x < 0 ? -1 : 1;
}

struct Point {
    double x, y;
    Point(double X = 0, double Y = 0): x(X), y(Y) {}
    Point operator + (Point B) { return Point(x + B.x, y + B.y); }
    Point operator - (Point B) { return Point(x - B.x, y - B.y); }
    Point operator * (double k) { return Point(x * k, y * k); }
    Point operator / (double k) { return Point(x / k, y / k); }
};

double Cross(Point A, Point B) {
    return A.x * B.y - A.y * B.x;
}

double Dot(Point A, Point B) {
    return A.x * B.x + A.y * B.y;
}

struct Line {
    Point p1, p2;
    Line(Point P1, Point P2): p1(P1), p2(P2) {}
};

bool Point_on_seg(Point p, Line v) {
    return sgn(Cross(p - v.p1, v.p2 - v.p1)) == 0
        && sgn(Dot(p - v.p1, p - v.p2)) <= 0;
}

bool Point_in_polygon(Point pt, Point* p, int n) {
    for (int i = 0; i < n; i++) {
        if (p[i].x == pt.x && p[i].y == pt.y) return 1;
    }
}
```

```

}

for (int i = 0; i < n; i++) {
    Line v = Line(p[i], p[(i + 1) % n]);
    if (Point_on_seg(pt, v)) return 1;
}

int num = 0;
for (int i = 0; i < n; i++) {
    int j = (i + 1) % n;
    int c = sgn(Cross(pt - p[j], p[i] - p[j]));
    int u = sgn(p[i].y - pt.y);
    int v = sgn(p[j].y - pt.y);

    if (c > 0 && u < 0 && v >= 0) num++;
    if (c < 0 && u >= 0 && v < 0) num--;
}
return num != 0;
}

```

3. 面积公式

3.1 三角形面积

三点 A, B, C 构成三角形面积公式:

$$S = \frac{1}{2} |\overrightarrow{AB} \times \overrightarrow{AC}|$$

```

#include<bits/stdc++.h>
using namespace std;

struct Point {
    double x, y;
};

double get(Point a, Point b, Point c) {
    return fabs((b.x - a.x) * (c.y - a.y) - (c.x - a.x) * (b.y - a.y)) / 2.0;
}

void solve() {
    Point a, b, c;
    cin >> a.x >> a.y;
    cin >> b.x >> b.y;
    cin >> c.x >> c.y;

    double ans = get(a, b, c);

    cout << fixed << setprecision(2) << ans << "\n";
}

int main() {

```

```

ios::sync_with_stdio(false);
cin.tie(0);

solve();
return 0;
}

```

3.2 多边形面积

如果多边形顶点按顺时针或逆时针顺序给出，就可以用鞋带公式：

$$S = \frac{1}{2} \left| \sum_{i=1}^n (x_i y_{i+1} - x_{i+1} y_i) \right|$$

其中当 $i=n$ 时， $i+1$ 回到 1 。

```

#include<bits/stdc++.h>
using namespace std;

struct Point {
    double x, y;
};

void solve() {
    int n;
    cin >> n;
    vector<Point> p(n + 1);

    for (int i = 1; i <= n; i++) {
        cin >> p[i].x >> p[i].y;
    }

    double ans = 0;
    for (int i = 1; i <= n; i++) {
        int j = (i % n) + 1;
        ans += p[i].x * p[j].y - p[j].x * p[i].y;
    }

    cout << fixed << setprecision(2) << fabs(ans) / 2.0 << "\n";
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);

    solve();
    return 0;
}

```

4. 凸包

4.1 Andrew 凸包

适用场景

- 求点集凸包
- 求凸包周长
- 为旋转卡壳做预处理

复杂度

- 排序: $O(n \log n)$
- 建凸包: $O(n)$

板子

```
#include<bits/stdc++.h>
using namespace std;

const int N = 1e5 + 10;

struct node {
    double x, y;
} f[N], s[N];

double cross(node a, node b, node c) {
    return (b.x - a.x) * (c.y - a.y) - (b.y - a.y) * (c.x - a.x);
}

double dis(node a, node b) {
    return sqrt((a.x - b.x) * (a.x - b.x) + (a.y - b.y) * (a.y - b.y));
}

bool cmp(node a, node b) {
    return a.x != b.x ? a.x < b.x : a.y < b.y;
}

void solve() {
    int n;
    cin >> n;
    for (int i = 1; i <= n; i++) cin >> f[i].x >> f[i].y;

    sort(f + 1, f + 1 + n, cmp);

    int top = 0;

    for (int i = 1; i <= n; i++) {
        while (top > 1 && cross(s[top - 1], s[top], f[i]) <= 0) top--;
        s[++top] = f[i];
    }
}
```

```

int t = top;

for (int i = n - 1; i >= 1; i--) {
    while (top > t && cross(s[top - 1], s[top], f[i]) <= 0) top--;
    s[++top] = f[i];
}

double res = 0;
for (int i = 1; i < top; i++) {
    res += dis(s[i], s[i + 1]);
}

cout << fixed << setprecision(2) << res << "\n";
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);

    solve();
    return 0;
}

```

说明

- `cross(...) <= 0`：会去掉共线点中间点，只保留端点
- 如果题目要求保留边上所有点，需要调整判断条件

5. 旋转卡壳

5.1 求凸包直径

旋转卡壳常用来求凸包上最远点对，也就是凸包直径。

思路一般是：

1. 先求凸包
2. 再在凸包上跑双指针

```

#include<bits/stdc++.h>
using namespace std;

const int N = 1e5 + 10;

struct node {
    double x, y;
} f[N], s[N];

double cross(node a, node b, node c) {
    return (b.x - a.x) * (c.y - a.y) - (b.y - a.y) * (c.x - a.x);
}

```

```

double dis(node a, node b) {
    return sqrt((a.x - b.x) * (a.x - b.x) + (a.y - b.y) * (a.y - b.y));
}

bool cmp(node a, node b) {
    return a.x != b.x ? a.x < b.x : a.y < b.y;
}

void solve() {
    int n;
    cin >> n;
    for (int i = 1; i <= n; i++) cin >> f[i].x >> f[i].y;

    sort(f + 1, f + 1 + n, cmp);

    int top = 0;
    for (int i = 1; i <= n; i++) {
        while (top > 1 && cross(s[top - 1], s[top], f[i]) <= 0) top--;
        s[++top] = f[i];
    }

    int t = top;
    for (int i = n - 1; i >= 1; i--) {
        while (top > t && cross(s[top - 1], s[top], f[i]) <= 0) top--;
        s[++top] = f[i];
    }

    n = top - 1;

    double res = 0;
    for (int i = 1, j = 2; i <= n; i++) {
        while (cross(s[i], s[i + 1], s[j]) < cross(s[i], s[i + 1], s[j % n + 1])) {
            j = j % n + 1;
        }
        res = max(res, max(dis(s[i], s[j]), dis(s[i + 1], s[j])));
    }

    cout << fixed << setprecision(0) << res * res << "\n";
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);

    solve();
    return 0;
}

```

注意

- 这份板子输出的是最远距离的平方
- 很多题会直接要求输出平方，这样可以减少精度误差

6.极角排序

6.1 适用场景

极角排序常用于：

- 向量按角度排序
- 扫描线 / 环形统计
- 同方向向量归类
- 极角双指针

6.2 常用写法

核心思想：

1. 先分上半平面和下半平面
2. 再用叉积比较先后顺序

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
using i128 = __int128;
using pll = pair<ll, ll>;

struct node {
    ll x, y, cnt;
};

ll gcdll(ll a, ll b) {
    if (a < 0) a = -a;
    if (b < 0) b = -b;
    return gcd(a, b);
}

int half(const node &a) {
    return (a.y > 0 || (a.y == 0 && a.x > 0)) ? 0 : 1;
}

i128 cross(const node &a, const node &b) {
    return (i128)a.x * b.y - (i128)a.y * b.x;
}

bool polar_cmp(const node &a, const node &b) {
    int ha = half(a), hb = half(b);
    if (ha != hb) return ha < hb;

    i128 cr = cross(a, b);
    if (cr != 0) return cr > 0;
```

```

    if (a.x != b.x) return a.x < b.x;
    return a.y < b.y;
}

void solve() {
    int n, q;
    cin >> n >> q;

    vector<pll> dir(n + 1);
    map<pll, ll> mp;

    for (int i = 1; i <= n; i++) {
        ll x, y;
        cin >> x >> y;
        ll g = gcdll(x, y);
        x /= g;
        y /= g;
        dir[i] = {x, y};
        mp[dir[i]]++;
    }

    vector<node> c;
    for (auto &it : mp) {
        c.push_back({it.first.first, it.first.second, it.second});
    }

    sort(c.begin(), c.end(), polar_cmp);

    int m = c.size();
    map<pll, int> pos;
    for (int i = 0; i < m; i++) {
        pos[{c[i].x, c[i].y}] = i;
    }

    vector<int> sc(n + 1);
    for (int i = 1; i <= n; i++) {
        sc[i] = pos[dir[i]];
    }

    vector<ll> pre(m + 1, 0);
    for (int i = 0; i < m; i++) {
        pre[i + 1] = pre[i] + c[i].cnt;
    }

    auto count_ccw = [&](int L, int R) -> long long {
        if (L <= R) return pre[R + 1] - pre[L];
        return (pre[m] - pre[L]) + (pre[R + 1] - pre[0]);
    };

    while (q--) {
        int a, b;
        cin >> a >> b;
    }
}

```

```

        int pa = sc[a], pb = sc[b];

        if (pa == pb) {
            cout << c[pa].cnt << "\n";
        } else {
            cout << count_ccw(pb, pa) << "\n";
        }
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);

    solve();
    return 0;
}

```

7. 题目型模板

7.1 CF993A Two Squares

这是一个经典入门题：判断两个正方形是否有交。

这份写法的核心思路是：

- 枚举一定范围内的整点
- 判断是否存在点同时落在两个正方形内

```

#include<bits/stdc++.h>
using namespace std;

int sgn(double x) {
    if (fabs(x) < 1e-9) return 0;
    return x < 0 ? -1 : 1;
}

struct Point {
    double x, y;
    Point(double X = 0, double Y = 0): x(X), y(Y) {}
    Point operator + (Point B) {
        return Point(x + B.x, y + B.y);
    }
    Point operator - (Point B) {
        return Point(x - B.x, y - B.y);
    }
    Point operator * (double k) {
        return Point(x * k, y * k);
    }
    Point operator / (double k) {
        return Point(x / k, y / k);
    }
}

```

```

    }
};

double Cross(Point A, Point B) {
    return A.x * B.y - A.y * B.x;
}

double Dot(Point A, Point B) {
    return A.x * B.x + A.y * B.y;
}

struct Line {
    Point p1, p2;
    Line(Point P1, Point P2): p1(P1), p2(P2) {}
};

bool Point_on_seg(Point p, Line v) {
    return sgn(Cross(p - v.p1, v.p2 - v.p1)) == 0 && sgn(Dot(p - v.p1, p - v.p2)) <= 0;
}

bool Point_in_polygon(Point pt, Point* p, int n) {
    for (int i = 0; i < n; i++) {
        if (p[i].x == pt.x && p[i].y == pt.y) return 1;
    }
    for (int i = 0; i < n; i++) {
        Line v = Line(p[i], p[(i + 1) % n]);
        if (Point_on_seg(pt, v)) return 1;
    }

    int num = 0;
    for (int i = 0; i < n; i++) {
        int j = (i + 1) % n;
        int c = sgn(Cross(pt - p[j], p[i] - p[j]));
        int u = sgn(p[i].y - pt.y);
        int v = sgn(p[j].y - pt.y);

        if (c > 0 && u < 0 && v >= 0) num++;
        if (c < 0 && u >= 0 && v < 0) num--;
    }
    return num != 0;
}

Point p1[4], p2[4];

void solve() {
    for (int i = 0; i < 4; i++) cin >> p1[i].x >> p1[i].y;
    for (int i = 0; i < 4; i++) cin >> p2[i].x >> p2[i].y;

    for (int i = -100; i <= 100; i++) {
        for (int j = -100; j <= 100; j++) {
            if (Point_in_polygon(Point(i, j), p1, 4) && Point_in_polygon(Point(i, j), p2, 4)) {
                cout << "YES";
                return;
            }
        }
    }
}

```

```

        }
    }
}
cout << "NO";
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);

    solve();
    return 0;
}

```

8. 圆的基础内容

8.1 圆的基本定义

在计算几何里，圆一般记录两个量：

- 圆心 `c`
- 半径 `r`

```

#include<bits/stdc++.h>
using namespace std;

int sgn(double x) {
    const double eps = 1e-9;
    if (fabs(x) < eps) return 0;
    return x < 0 ? -1 : 1;
}

struct Point {
    double x, y;
    Point(double X = 0, double Y = 0): x(X), y(Y) {}

    Point operator + (const Point &B) const {
        return Point(x + B.x, y + B.y);
    }
    Point operator - (const Point &B) const {
        return Point(x - B.x, y - B.y);
    }
    Point operator * (double k) const {
        return Point(x * k, y * k);
    }
    Point operator / (double k) const {
        return Point(x / k, y / k);
    }
};

double Cross(Point A, Point B) {

```

```

        return A.x * B.y - A.y * B.x;
    }

    double Dot(Point A, Point B) {
        return A.x * B.x + A.y * B.y;
    }

    double Length(Point A) {
        return sqrt(Dot(A, A));
    }

    double dis(Point a, Point b) {
        return Length(a - b);
    }

    struct Circle {
        Point c;
        double r;
        Circle(Point C = Point(), double R = 0): c(C), r(R) {}
    };

```

8.2 点和圆的位置关系

设点为 p ，圆为 $O(c, r)$ 。

判断方法很直接：

- $|p-c| < r$ ：点在圆内
- $|p-c| = r$ ：点在圆上
- $|p-c| > r$ ：点在圆外

```

// 0: 圆外
// 1: 圆上
// 2: 圆内
int Point_circle_relation(Point p, Circle O) {
    double d = dis(p, O.c);
    int t = sgn(d - O.r);
    if (t < 0) return 2;
    if (t == 0) return 1;
    return 0;
}

```

8.3 点在直线上的投影

圆和直线求交时，常常要先求圆心到直线的投影点，所以这个函数建议和圆放在一起。

设点为 p ，直线经过 a, b 。

```
Point Projection(Point p, Point a, Point b) {
    Point v = b - a;
    return a + v * (Dot(p - a, v) / Dot(v, v));
}
```

说明

这个点就是 `p` 在直线 `ab` 上的垂足。

8.4 点到直线距离

设点为 `p`，直线经过 `a`, `b`。

公式是：

$$\frac{|(b - a) \times (p - a)|}{|b - a|}$$

```
double Distance_to_line(Point p, Point a, Point b) {
    Point v1 = b - a;
    Point v2 = p - a;
    return fabs(Cross(v1, v2)) / Length(v1);
}
```

8.5 直线和圆的位置关系

设直线经过 `a`, `b`，圆为 `O(c, r)`。

分类如下：

- 距离 `> r`：相离
- 距离 `= r`：相切
- 距离 `< r`：相交

```
// 0: 相离
// 1: 相切
// 2: 相交
int Line_circle_relation(Point a, Point b, Circle O) {
    double d = Distance_to_line(O.c, a, b);
    int t = sgn(d - O.r);
    if (t > 0) return 0;
    if (t == 0) return 1;
    return 2;
}
```

8.6 圆和圆的位置关系

设两个圆分别为 $A(c_1, r_1)$ 、 $B(c_2, r_2)$ ，圆心距为 d 。

分类：

- $d > r_1 + r_2$ ：外离
- $d = r_1 + r_2$ ：外切
- $|r_1 - r_2| < d < r_1 + r_2$ ：相交
- $d = |r_1 - r_2|$ ：内切
- $d < |r_1 - r_2|$ ：内含
- $d = 0$ 且 $r_1 = r_2$ ：重合

```
// -1: 重合
// 0: 外离
// 1: 外切
// 2: 相交
// 3: 内切
// 4: 内含
int Circle_circle_relation(Circle A, Circle B) {
    double d = dis(A.c, B.c);

    if (sgn(d) == 0 && sgn(A.r - B.r) == 0) return -1;

    if (sgn(d - (A.r + B.r)) > 0) return 0;
    if (sgn(d - (A.r + B.r)) == 0) return 1;
    if (sgn(d - fabs(A.r - B.r)) > 0) return 2;
    if (sgn(d - fabs(A.r - B.r)) == 0) return 3;
    return 4;
}
```

8.7 直线与圆的交点

思路

设直线为 ab ，圆为 $O(c, r)$ 。

做法：

1. 先求圆心 c 在直线 ab 上的投影点 q
2. 计算 q 到圆心的距离 d
3. 比较 d 和 r
4. 若有两个交点，则从 q 沿直线方向左右各走一段

```
int Line_circle_intersection(Point a, Point b, Circle O, vector<Point> &sol) {
    sol.clear();

    Point q = Projection(O.c, a, b);
    double d = Length(q - O.c);
```

```

int t = sgn(d - 0.r);
if (t > 0) return 0;

Point v = b - a;
v = v / Length(v);

if (t == 0) {
    sol.push_back(q);
    return 1;
}

double len = sqrt(0.r * 0.r - d * d);
sol.push_back(q - v * len);
sol.push_back(q + v * len);
return 2;
}

```

返回值

- 0: 无交点
- 1: 一个交点
- 2: 两个交点

8.8 圆与圆的交点

思路

设两个圆为 $A(c1, r1)$ 、 $B(c2, r2)$ 。

步骤:

1. 先判断两圆是否有交点
2. 设圆心距为 d
3. 求出公共弦中点 mid
4. 沿着垂直方向展开, 得到两个交点

```

int Circle_circle_intersection(Circle A, Circle B, vector<Point> &sol) {
    sol.clear();

    double d = Length(B.c - A.c);

    if (sgn(d) == 0 && sgn(A.r - B.r) == 0) return -1;
    if (sgn(d - (A.r + B.r)) > 0) return 0;
    if (sgn(d - fabs(A.r - B.r)) < 0) return 0;
    if (sgn(d) == 0) return 0;

    double x = (d * d + A.r * A.r - B.r * B.r) / (2 * d);
    double h2 = A.r * A.r - x * x;

    if (h2 < 0) h2 = 0;
}

```

```

Point v = (B.c - A.c) / d;
Point mid = A.c + v * x;

if (sgn(h2) == 0) {
    sol.push_back(mid);
    return 1;
}

double h = sqrt(h2);
Point per = Point(-v.y, v.x);

sol.push_back(mid + per * h);
sol.push_back(mid - per * h);
return 2;
}

```

返回值

- `-1`：无穷多个交点，两个圆重合
- `0`：无交点
- `1`：一个交点
- `2`：两个交点

说明

这里的

```
if (h2 < 0) h2 = 0;
```

是为了防精度误差。理论上切圆时 `h2 = 0`，但浮点数有时会算成一个很小的负数。

8.9 圆面积与周长

```

const double pi = acos(-1.0);

double Circle_area(Circle O) {
    return pi * O.r * O.r;
}

double Circle_perimeter(Circle O) {
    return 2.0 * pi * O.r;
}

```

8.10 扇形面积

如果题目里涉及圆心角，扇形面积也比较常用。

设圆心角为 `theta`，单位是弧度。

```
double Sector_area(Circle O, double theta) {  
    return 0.5 * O.r * O.r * theta;  
}
```

说明

这个公式默认 `theta` 是弧度，不是角度。如果题目给的是角度，需要先转成弧度。

六、杂板子整理

1. 三分

核心结论

若函数在区间内满足单峰性：

- 求最小值时，若 $f(m1) < f(m2)$ ，则答案在 $[1, m2]$
- 否则答案在 $[m1, r]$

其中：

```
m1 = (2 * l + r) / 3;  
m2 = (l + 2 * r) / 3;
```

注意点

- 前提是函数具有单峰性，否则不能乱用
- 实数三分一般迭代 $80 \sim 100$ 次就够
- 输出极值时通常输出 $f(l)$ 或 $f((l + r) / 2)$
- 若是整数三分，最后要对收缩后的短区间暴力比较

模板：实数三分求最小值

```
#include<bits/stdc++.h>  
using namespace std;  
  
void solve() {  
    int n;  
    cin >> n;  
  
    vector<double> a(n + 1), b(n + 1), c(n + 1);  
    for (int i = 1; i <= n; i++) {  
        cin >> a[i] >> b[i] >> c[i];  
    }
```

```

}

auto f = [&](double x) {
    double ans = -1e100;
    for (int i = 1; i <= n; i++) {
        ans = max(ans, a[i] * x * x + b[i] * x + c[i]);
    }
    return ans;
};

double l = 0, r = 1000;
for (int i = 1; i <= 100; i++) {
    double m1 = (2 * l + r) / 3;
    double m2 = (l + 2 * r) / 3;
    if (f(m1) < f(m2)) r = m2;
    else l = m1;
}

cout << fixed << setprecision(4) << f(l) << "\n";
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);
    int _T = 1;
    cin >> _T;
    while (_T--) solve();
    return 0;
}

```

模板：整数三分

```

while (r - l > 3) {
    int m1 = l + (r - l) / 3;
    int m2 = r - (r - l) / 3;
    if (f(m1) < f(m2)) l = m1;
    else r = m2;
}
int ans = l;
for (int i = l; i <= r; i++) {
    if (f(i) > f(ans)) ans = i;
}

```

2. 对拍随机数

```

#include <bits/stdc++.h>

using namespace std;

mt19937_64 rnd(chrono::steady_clock::now().time_since_epoch().count());

```

```

int rand(int l, int r) { //l: 随机数的最小值（闭区间）r: 随机数的最大值（闭区间）
    uniform_int_distribution<int> dist(l, r);
    return dist(rnd);
}

int main() {
    int n = rand(0, 9);
    cout << n << "\n";
    for (int i = 1; i <= n; i++) {
        int l = rand(0, 10);
        int r = rand(l, 10);
        cout << l << " " << r << "\n";
    }
    return 0;
}

```

3. 对拍器

```

#include <iostream>
#include <cstdio>
#include <cstdlib>
#include <chrono>

using namespace std;
using namespace std::chrono;

// 利用宏定义自动识别系统平台，配置对应的运行命令
#ifdef _WIN32
const string GEN_CMD = "data.exe > in.txt";
const string TEST_CMD = "test.exe < in.txt > test_out.txt";
const string BAOLI_CMD = "baoli.exe < in.txt > baoli_out.txt";
const string DIFF_CMD = "fc test_out.txt baoli_out.txt > nul";
#else
const string GEN_CMD = "./data > in.txt";
const string TEST_CMD = "./test < in.txt > test_out.txt";
const string BAOLI_CMD = "./baoli < in.txt > baoli_out.txt";
const string DIFF_CMD = "diff -w test_out.txt baoli_out.txt > /dev/null";
#endif

int main() {
    int ok = 0, n = 100; // n 为测试数据的组数

    cout << "开始对拍..." << endl;

    for(int i = 1; i <= n; i++){
        // 1. 生成随机数据
        system(GEN_CMD.c_str());

        // 2. 运行待测试代码
        system(TEST_CMD.c_str());

        // 3. 运行暴力/正确代码，并使用 chrono 精确计时

```

```

auto begin = steady_clock::now();
system(BAOLI_CMD.c_str());
auto end = steady_clock::now();

// 获取真实流逝的毫秒数
auto t = duration_cast<milliseconds>(end - begin).count();

// 4. 比对输出结果
if(system(DIFF_CMD.c_str())){
    printf("测试点%d Wrong Answer\n", i);
    break; // 遇到 WA 直接退出, 保留 in.txt 现场方便 Debug
} else if(t > 1000){ // 假设时间限制为 1000ms
    printf("暴力/测试代码 Time Limit Exceeded (用时 %lld ms)\n", t);
    break; // 建议超时也保留现场
} else {
    printf("测试点%d Accepted 用时 %lld ms\n", i, t);
    ok++;
}
}

printf("\n测试结束。 \n");
double res = 100.0 * ok / n;
printf("共 %d 组测试数据, AC数据 %d 组。 得分 %.11f。 \n", n, ok, res);

return 0;
}

```

七、贪心

1. 区间问题

1.1 区间合并

题型特征

给若干区间, 要求把所有有交集的区间合并, 求合并后的区间个数或合并后的结果。

贪心策略

按左端点从小到大排序。

核心思路

维护当前合并区间的右端点 `ed` :

- 若新区间左端点 $l \leq ed$, 说明有交集, 更新 $ed = \max(ed, r)$
- 否则说明出现新区间, 答案加一, 并重置 `ed`

模板

```

#include<bits/stdc++.h>
using namespace std;

```

```

struct Seg {
    int l, r;
    bool operator < (const Seg &t) const {
        return l < t.l;
    }
};

void solve() {
    int n;
    cin >> n;
    vector<Seg> segs(n);
    for (int i = 0; i < n; i++) cin >> segs[i].l >> segs[i].r;

    sort(segs.begin(), segs.end());

    int cnt = 1;
    int ed = segs[0].r;

    for (int i = 1; i < n; i++) {
        if (segs[i].l <= ed) ed = max(ed, segs[i].r);
        else {
            cnt++;
            ed = segs[i].r;
        }
    }

    cout << cnt << "\n";
}

```

1.2 区间选点

题型特征

给若干闭区间，选尽量少的点，使每个区间至少包含一个点。

贪心策略

按右端点从小到大排序，优先选当前区间的右端点。

核心思路

因为右端点越小，越能兼顾后面的区间。
所以每次都选当前最早结束区间的右端点。

模板

```

#include<bits/stdc++.h>
using namespace std;

struct Seg {
    int l, r;
    bool operator < (const Seg &t) const {

```

```

        return r < t.r;
    }
};

void solve() {
    int n;
    cin >> n;
    vector<Seg> segs(n);
    for (int i = 0; i < n; i++) cin >> segs[i].l >> segs[i].r;

    sort(segs.begin(), segs.end());

    int cnt = 1;
    int ed = segs[0].r;

    for (int i = 1; i < n; i++) {
        if (segs[i].l <= ed) continue;
        ed = segs[i].r;
        cnt++;
    }

    cout << cnt << "\n";
}

```

1.3 最大不相交区间数量

题型特征

给若干区间，要求选出尽可能多的区间，使它们两两不相交。

贪心策略

按右端点从小到大排序。

核心思路

这题和“区间选点”本质一样。

每次优先选右端点最小的区间，这样能给后面留下最多空间。

模板

```

#include<bits/stdc++.h>
using namespace std;

struct Seg {
    int l, r;
    bool operator < (const Seg &t) const {
        return r < t.r;
    }
};

void solve() {
    int n;

```

```

cin >> n;
vector<Seg> segs(n);
for (int i = 0; i < n; i++) cin >> segs[i].l >> segs[i].r;

sort(segs.begin(), segs.end());

int cnt = 1;
int ed = segs[0].r;

for (int i = 1; i < n; i++) {
    if (segs[i].l <= ed) continue;
    ed = segs[i].r;
    cnt++;
}

cout << cnt << "\n";
}

```

1.4 区间分组

题型特征

给若干区间，要求分成尽量少的组，使每组内部区间两两不相交。

贪心策略

按左端点从小到大排序，再用小根堆维护每组当前最小的右端点。

核心思路

- 堆顶表示当前最早结束的那一组
- 如果当前区间左端点 `l <= heap.top()`，说明和最早结束的那组也有冲突，只能新开一组
- 否则可以放进这组，弹出旧右端点，加入新区间右端点

模板

```

#include<bits/stdc++.h>
using namespace std;

struct Seg {
    int l, r;
    bool operator < (const Seg &t) const {
        return l < t.l;
    }
};

void solve() {
    int n;
    cin >> n;
    vector<Seg> segs(n);
    for (int i = 0; i < n; i++) cin >> segs[i].l >> segs[i].r;
}

```

```

sort(segs.begin(), segs.end());

priority_queue<int, vector<int>, greater<int>> heap;
heap.push(segs[0].r);

for (int i = 1; i < n; i++) {
    if (segs[i].l <= heap.top()) {
        heap.push(segs[i].r);
    } else {
        heap.pop();
        heap.push(segs[i].r);
    }
}

cout << heap.size() << "\n";
}

```

1.5 区间覆盖

题型特征

给定目标区间 `[st, ed]`，从若干区间中选尽量少的区间把它完全覆盖。

贪心策略

按左端点从小到大排序，每次在所有能接上当前 `st` 的区间中，选右端点最远的那个。

核心思路

设当前要覆盖到的位置是 `st`：

- 找所有满足 `l <= st` 的区间
- 在这些区间中选 `r` 最大的
- 这样一步尽量向右扩展最多
- 若最大 `r` 仍然小于 `st`，说明无法覆盖

模板

```

#include<bits/stdc++.h>
using namespace std;

const int INF = 2e9 + 10;

struct Seg {
    int l, r;
    bool operator < (const Seg &t) const {
        return l < t.l;
    }
};

void solve() {
    int st, ed, n;

```

```

cin >> st >> ed >> n;

vector<Seg> segs(n);
for (int i = 0; i < n; i++) cin >> segs[i].l >> segs[i].r;

sort(segs.begin(), segs.end());

int cnt = 0;

for (int i = 0; i < n; i++) {
    int j = i;
    int r = -INF;

    while (j < n && segs[j].l <= st) {
        r = max(r, segs[j].r);
        j++;
    }

    if (r < st) break;

    cnt++;
    st = r + 1;

    if (st > ed) {
        cout << cnt << "\n";
        return;
    }

    i = j - 1;
}

cout << -1 << "\n";
}

```

2. 反悔贪心

适用场景

有一批决策按顺序出现：

- 当前先贪心选一个
- 以后发现有更优选择时，允许撤销之前较差的选择
- 典型维护结构通常是优先队列、平衡树

经典思路

用一个堆维护“当前已选方案中最差的那个”。若新来的更优，就把最差的弹掉，再换成当前更优的。

做法理解

最小堆里维护“之前买入的股价”。若当前价格 x 比堆顶更大，说明把之前更低价买的那股放到今天卖更优：

- 收益增加 $x - pq.top()$
- 弹出旧买入价

- 插入当前价，表示这次卖出后仍可视为“在今天重新买回一股”，以便后续继续反悔

模板

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;

void solve() {
    int n;
    cin >> n;

    priority_queue<ll, vector<ll>, greater<ll>> pq;
    ll ans = 0;

    for (int i = 1; i <= n; i++) {
        ll x;
        cin >> x;
        if (!pq.empty() && pq.top() < x) {
            ans += x - pq.top();
            pq.pop();
            pq.push(x);
        }
        pq.push(x);
    }

    cout << ans << "\n";
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);
    int _T = 1;
    while (_T--) solve();
    return 0;
}
```

八、数值处理

1. __int128

以A+B为例

```
#include<cstdio>
#include<cctype>

using namespace std;
using i128 = __int128;

i128 read()
```

```

{
    i128 x = 0, f = 1;

    char c = getchar();
    while(!isdigit(c))
    {
        if(c == '-') f = -1;
        c = getchar();
    }

    while(isdigit(c))
    {
        x = x * 10 + c - '0';
        c = getchar();
    }

    return x * f;
}

void print(i128 x)
{
    if(x < 0)
    {
        putchar('-');
        x = -x;
    }

    if (x > 9) print(x / 10);
    putchar(x % 10 + '0');
}

int main()
{
    i128 x = read();
    i128 y = read();

    print(x + y);

    return 0;
}

```

2. long double

以A+B为例

```

#include<cstdio>

using namespace std;

int main()
{
    long double a, b;

    scanf("%Lf%Lf", &a, &b);
    printf("%.0Lf", a + b);

    return 0;
}

```

3. 高精度

```

#include<iostream>
#include<string>
#include<vector>
#include<algorithm>
using namespace std;

// 字符串转倒序数组
vector<int> tfm(string& s) {
    vector<int> res;
    for (int i = s.size() - 1; i >= 0; i--) {
        res.push_back(s[i] - '0');
    }
    return res;
}

// 输出高精度
void print(vector<int>& num) {
    for (int i = num.size() - 1; i >= 0; i--) {
        cout << num[i];
    }
    cout << "\n";
}

// 高精度加法
vector<int> add(vector<int>& A, vector<int>& B) {
    if (A.size() < B.size()) return add(B, A);
    vector<int> C;
    int t = 0;
    for (int i = 0; i < A.size(); i++) {
        t += A[i];
        if (i < B.size()) t += B[i];
        C.push_back(t % 10);
        t /= 10;
    }
    if (t) C.push_back(t);
}

```

```

    return C;
}

// 比较字符串大小: 是否 a < b
bool cmp(string& a, string& b) {
    if (a.size() == b.size()) return a < b;
    return a.size() < b.size();
}

// 高精度减法, 要求 A >= B
vector<int> sub(vector<int>& A, vector<int>& B) {
    vector<int> C;
    int t = 0;
    for (int i = 0; i < A.size(); i++) {
        t += A[i];
        if (i < B.size()) t -= B[i];
        C.push_back((t + 10) % 10);
        t = t < 0 ? -1 : 0;
    }
    while (C.size() > 1 && C.back() == 0) C.pop_back();
    return C;
}

// 高精度 * 低精度
vector<int> mul(vector<int>& A, int b) {
    vector<int> C;
    int t = 0;
    for (int i = 0; i < A.size() || t; i++) {
        if (i < A.size()) t += A[i] * b;
        C.push_back(t % 10);
        t /= 10;
    }
    while (C.size() > 1 && C.back() == 0) C.pop_back();
    return C;
}

// 高精度 * 高精度
vector<int> mul(vector<int>& A, vector<int>& B) {
    vector<int> C(A.size() + B.size() + 10);
    for (int i = 0; i < A.size(); i++) {
        for (int j = 0; j < B.size(); j++) {
            C[i + j] += A[i] * B[j];
        }
    }
    int t = 0;
    for (int i = 0; i < C.size(); i++) {
        t += C[i];
        C[i] = t % 10;
        t /= 10;
    }
    while (C.size() > 1 && C.back() == 0) C.pop_back();
    return C;
}

```

```

// 高精度 / 低精度, r 为余数
vector<int> div(vector<int>& A, int b, int& r) {
    vector<int> C;
    for (int i = A.size() - 1; i >= 0; i--) {
        r *= 10;
        r += A[i];
        C.push_back(r / b);
        r %= b;
    }
    reverse(C.begin(), C.end());
    while (C.size() > 1 && C.back() == 0) C.pop_back();
    return C;
}

```

4. 文件读入

```

int main() {
    freopen("in.txt", "r", stdin);
    freopen("out.txt", "w", stdout);

    ios::sync_with_stdio(false);
    cin.tie(0);

    solve();
    return 0;
}

```